
NEOLAF: A Neuro-Symbolic Cognitive Architecture for Experiential Learning

Richard Jiarui Tong^{1,4,5}, Cassie Chen Cao², Timothy Xueqian Lee³, Ray Wan¹, Feiyue Wang¹, and Xiangen Hu⁵

¹Macau University of Science and Technology, Macao

²University of Sheffield, Sheffield, UK

³National Institute of Education, Singapore

⁴IEEE Artificial Intelligent Standards Committee, IEEE, NJ, USA

⁵NEOLAF Inc., NJ, USA

Abstract

This paper addresses a fundamental challenge in artificial intelligence: moving beyond static, pre-trained models toward agents capable of continuous, experiential learning after their initial deployment. We introduce NEOLAF (Neuro-Symbolic Cognitive Architecture for Experiential Learning), a novel framework designed to enable AI to learn and adapt throughout its lifecycle, much like humans and animals do. Our central thesis is built upon four core innovations. First, we propose a neuro-symbolic agentic architecture featuring dual memory and dual computation systems, specifically engineered for continuous "after-birth" learning. Second, we introduce the recursive KSTAR (Knowledge-Situation-Task-Action-Result) process, a systematic cycle for transforming any experience into structured knowledge. Third, we leverage a Large Language Model (LLM) as a neural bridge, using natural language as the lingua franca to mediate between the system's symbolic and neural components. Fourth, the entire learning process is formalized using category theory, providing a mathematically rigorous foundation for analysis and implementation.

Part I

Introduction and Inspiration

1 Introduction and Overview

1.1 The Motivation: Towards Self-Improvable and Grounded AI Agents

The advent of large language models (LLMs) has marked a significant milestone in artificial intelligence, yet it has also illuminated the next great frontier: the transition from static, pre-trained repositories of knowledge to dynamic, self-improvable agents capable of continuous learning. Today's models, while possessing vast information, are akin to encyclopedias; they can answer questions based on the text they have seen but cannot truly learn from a new conversation, a novel experience, or direct instruction. They lack the fundamental mechanism that underpins all biological intelligence: the ability to learn "after birth."

This work is motivated by the need to create an AI that can learn as humans and animals do—continuously, experientially, and interactively. Our primary goal is to design an agent that can be taught, not just prompted; one that can assist in complex domains like education by building a base of knowledge that is not merely recalled, but is validated, grounded, and understood. For an

AI to be a trustworthy tutor, collaborator, or assistant, its knowledge cannot be an opaque artifact of its training data. It must be able to explain its reasoning by tracing it back to verifiable experiences, making the agent both explainable and grounded. This paper introduces a cognitive architecture designed to meet this challenge, enabling agents that can genuinely improve and adapt throughout their operational lives.

1.2 Our Thesis: Four Core Innovations for a Learning Agent

To achieve this vision, we propose the Never-Ending Open Learning Agent Framework (NEOLAF), an architecture built upon four core innovations that, together, enable continuous experiential learning:

1. **A Neuro-Symbolic Agentic Architecture:** We construct a cognitive architecture with dual memory and dual computation capabilities. This structure is specifically designed to allow an AI agent to learn continuously after its initial deployment, providing distinct systems for intuitive, neural processing and deliberate, symbolic reasoning.
2. **The Recursive KSTAR Process:** We propose the KSTAR (Knowledge, Situation, Task, Action, Result) process as a systematic and structured framework that turns any experience into knowledge. It operates as a recursive cycle of observing, planning, doing, and learning, providing a universal engine for experiential updates.
3. **The LLM as a Neural Bridge:** We use the LLM/Transformer as the central neural engine that bridges the symbolic and neural worlds. In this role, it leverages natural language as the "lingua franca" for representational transformation and mapping, translating between unstructured human intent and the structured symbolic representations required for deliberate planning and reasoning.
4. **A Formal Foundation in Category Theory:** We apply the principles of category theory to formalize this entire learning process. This mathematical language provides a rigorous and unambiguous foundation for analyzing the relationships between knowledge, experience, and learning, ensuring the architecture is sound and well-defined.

1.3 Key Research Questions

This thesis leads us to address three central research questions:

[RQ1]: How can we design an architecture that systematically grounds abstract, representational knowledge in verifiable, real-world experience?

[RQ2]: In a modern learning-focused system, what is the most effective role for a large language model, moving beyond a simple input-output function?

[RQ3]: How can we formalize the process of experiential learning to ensure it is robust, consistent, and computationally tractable?

1.4 Overview of the Paper’s Structure

This paper is organized into five parts to guide the reader from the foundational motivations to the empirical results.

- **Part I** establishes the theoretical and philosophical landscape, drawing from Parallel Intelligence, the history of cognitive architectures, and modern AI to motivate NEOLAF’s unique design.
- **Part II** details the formal foundation of the architecture, using the language of category theory to define the core principles of the system in a top-down, cascading manner.
- **Part III** presents the anatomy and physiology of the NEOLAF agent, describing its static architectural components, its hierarchical memory systems, and the dynamic, step-by-step process of a single learning cycle.
- **Part IV** provides the concrete implementation details and the empirical validation of the framework through a case study in advanced mathematical reasoning.
- **Part V** concludes with a synthesis of our findings, a discussion of the implications, and a vision for the future of self-improving, symbiotic AI agents.

2 Related Work and Inspirations

2.1 The Parallel Intelligence Mandate: An Agent as a Being in Parallel Worlds

The philosophical underpinning of NEOLAF is deeply informed by the theory of Parallel Intelligence, which posits that intelligent systems evolve through the continuous interaction between real-world processes and parallel artificial or virtual systems [8]. This paradigm provides a powerful mandate for AI: to build agents that are, in essence, beings operating in parallel worlds. The essence of agency, from this perspective, is defined by the ability to form rich, predictive models of the world at the informational or virtual level (**Representation** $\mathbb{R}$) and then to validate, ground, and refine these models through direct, time-bound interaction with the physical world (**Substrate** $\mathbb{S}$).

In the context of Parallel Education, this manifests as creating a "digital twin" of the learning process, where AI agents can run computational experiments on "mental action plans" in a virtual space to guide real-world action [9]. NEOLAF internalizes this concept. Its cognitive cycle—where it forms an *Expected Result* within its internal representational world and then compares it to the *Actual Result* observed from executing actions against the substrate—is a direct implementation of this parallel-world interaction. This continuous loop of virtual prediction and physical validation is the core mechanism by which the agent learns and its intelligence co-evolves with its experience.

2.2 Lineage from Cognitive Architectures

NEOLAF builds upon a rich history of cognitive architectures that sought to create unified theories of cognition. These foundational systems, primarily symbolic in nature, established key principles of agent design.

- **SOAR (State, Operator And Result):** Developed by Laird, Newell, and Rosenbloom, SOAR models all goal-oriented tasks as operating within a problem space, using production rules to transition between states [3]. Its cycle of operator proposal, selection, and application, along with its "chunking" mechanism for learning, informs NEOLAF's structured, state-based approach to problem-solving.
- **ACT-R (Adaptive Control of Thought—Rational):** Developed by John R. Anderson, ACT-R provides an integrated theory that distinguishes between a declarative memory (representing facts as 'chunks') and a procedural memory (representing skills as 'production rules') [1]. NEOLAF's dual M1 (neural) and M2 (symbolic) memory systems are a direct evolution of this principle, modernizing the implementation with neural and symbolic technologies.

While these architectures laid the groundwork, they were often limited by the "brittleness" of purely symbolic systems and the knowledge acquisition bottleneck. NEOLAF addresses these challenges by integrating a powerful neural engine at its core.

2.3 Inspirations from the Neural Revolution

Two key developments from the modern neural revolution are central to NEOLAF's design: the Reinforcement Learning paradigm and the Transformer architecture.

2.3.1 The Reinforcement Learning Paradigm

Reinforcement Learning (RL) formalizes the process of learning through interaction. An agent learns an optimal policy by taking actions in an environment to maximize a cumulative reward signal [6]. This process is typically modeled as a Markov Decision Process (MDP), defined by a tuple $\langle S, A, P, R, \gamma \rangle$ (States, Actions, Transition Probabilities, Rewards, Discount Factor). We posit that the KSTAR cycle is isomorphic to this paradigm: the agent observes a *Situation* (State), formulates an *Action Plan* (Action), and the comparison of the *Actual Result* to the *Expected Result* serves as an intrinsic *Reward* signal that updates its internal *Knowledge* (Policy).

2.3.2 The Transformer as a Universal Compiler

The Transformer architecture, with its core self-attention mechanism, provides a foundational tool for representational transformation [7]. We view it as a universal, data-driven compiler. It excels at

mapping sequences of symbols from one representation to another (e.g., natural language to structured code), making it the ideal engine for bridging the neural and symbolic worlds. Its ability to process context and generate coherent, structured outputs based on vast pre-trained knowledge allows it to serve as the engine for the f_{nn} functor in our architecture, translating high-level tasks into concrete, symbolic action plans.

2.4 The State of Modern Neuro-Symbolic AI

Neuro-symbolic AI seeks to combine the strengths of neural networks (learning, pattern recognition) with symbolic systems (reasoning, interpretability) [2]. Many modern approaches, such as the Neuro-Symbolic Concept Learner (NS-CL) [4], often use neural components for perception (e.g., parsing a visual scene) and pass the resulting symbolic representation to a reasoning module. NEOLAF contributes to this field with a "neural-first" approach. Instead of merely using the neural network for perception, we leverage the LLM as the primary *reasoning and planning engine* that generates symbolic structures and programs as its output, which are then validated and executed by a deterministic symbolic core.

2.5 Inspiration from Never-Ending Learning

The vision for NEOLAF’s continuous learning is significantly inspired by systems like NELL (Never-Ending Language Learner) [5]. NELL was designed to continuously read the web to extract structured facts and autonomously build a growing symbolic knowledge base. While facing challenges in maintaining consistency, its core ambition—to create a system that learns 24/7 from vast, unstructured data and improves over time—is a direct influence on the "Never-Ending" principle of NEOLAF. NEOLAF adapts this vision to an agentic context, where learning comes not just from passive reading, but from active, goal-directed experience through the KSTAR cycle.

Part II

Theoretical and Formal Foundation

3 A Cascading Formalism for a Learning Agent

3.1 A Metaphysical Starting Point: Information vs. Substrate

At the core of our framework lies a dual representational ontology that distinguishes between the physical *substrate* of reality and the abstract realm of *Information*. This distinction is crucial for understanding how an intelligent agent operates simultaneously in the world of matter and energy, and in the world of information and meaning. We formalize this distinction using the language of category theory, by introducing two fundamental categories: the Substrate category $\mathbb{S}$ and the Information category $\mathbb{R}$. Intuitively, $\mathbb{S}$ captures the physical domain in which the agent is embodied, while $\mathbb{R}$ captures the informational domain in which the agent’s knowledge, thoughts, and computations reside.

In the Substrate category $\mathbb{S}$, the objects correspond to concrete physical entities or states (for example, masses, energy distributions, sensor readings as physical quantities), and the morphisms represent physical transformations or spatiotemporal relations between these states. The most fundamental morphisms in $\mathbb{S}$ are those dictated by physics itself: Time and Space. Time-evolution and spatial relations govern how substrate states change and relate; for instance, a morphism in $\mathbb{S}$ might represent the progression of the environment from one moment to the next under certain actions, or the movement of an agent through space. In this sense, $\mathbb{S}$ encapsulates the dynamic, temporal, and spatially-grounded nature of the real world. Every physical entity, including the hardware that an AI agent runs on, is an object in $\mathbb{S}$, and any physical process or causal interaction is a morphism in $\mathbb{S}$.

3.2 Deconstructing Information: Representation and Computation

The Information category, $\mathbb{R}$, is where cognition happens. We define its primary objects as **Representations**—the structures that stand for something else. These include symbols, data structures, vectors in a neural network, sentences in an internal language, or any other form of meaningful information an agent can hold.

The primary morphisms are **Computations**—the processes that manipulate these representations. Two especially important kinds of morphisms are (1) representational transformations, which convert or translate one representational structure into another (for example, parsing a sentence into a logical form, or encoding raw data into a latent vector), and (2) compositions of representations or operations, which combine smaller pieces of information into larger structures or sequences (for example, composing concepts into a complex idea, or chaining functions in a program). Unlike the substrate domain, which is inherently governed by temporal progression, the informational domain is in a sense *timeless*: an abstract piece of information (say a mathematical truth or a memory record) does not intrinsically age or change over time on its own; it must be actively transformed or updated by processes (morphisms) within $\mathbb{R}$. This “timeless” nature of information means that knowledge can, in principle, be preserved indefinitely, copied, or processed out of the flow of physical time—although, of course, any physical instantiation of information (in hardware, brain tissue, etc.) is still subject to substrate constraints. In summary, $\mathbb{R}$ captures the domain of meaning, interpretation, and computation, which while always ultimately implemented on a physical substrate, obeys its own structural and logical laws distinct from the laws of physics. The connection between the information and the substrate world is that any computation (morphism) in the information space requires energy in the substrate world because information is encoded (resides in) substrate carrier (medium). The change of Any representational state (aka, the object in the information Category) requires the material change on information carrier, hence it would require energy.

It is important to note that information, as treated in $\mathbb{R}$, is not an objective, disembodied substance; it is always *about* something and for some purpose. Information is inherently **intentional** and must be **subjective**—its significance is defined relative to the interpreting agent. An agent imbues raw data with meaning by interpreting it, placing it into context with its existing knowledge. Thus, although we distinguish $\mathbb{S}$ and $\mathbb{R}$ for clarity, they are deeply intertwined: information requires a physical medium to exist (there is no representation without some substrate to encode it), and conversely, intelligent physical processes (like an agent’s actions) are guided by information.

Placeholder for a diagram illustrating the dual ontological framework: a lower layer for the Substrate (physical world, with time/space arrows) and an upper layer for Representation (informational structures, with transformation arrows), and the relationships between them.

Figure 1: Illustration of the dual ontology in NEOLAF: the physical substrate domain $\mathbb{S}$ (governed by time and space) versus the representational domain $\mathbb{R}$ (governed by informational transformations). The agent exists at the intersection, interpreting substrate events as information and enacting informational plans into substrate actions.

Within this ontological framework, an intelligent agent is understood as an entity that straddles both categories. The agent’s body, sensors, and effectors reside in the substrate $\mathbb{S}$, while the agent’s mind—its knowledge, beliefs, goals, and internal models—resides in the representation category $\mathbb{R}$. Intelligence emerges from the interplay between these realms: the agent perceives the substrate world and encodes those perceptions as internal representations, and it uses its internal representations to forecast the future and decide on action choices that it then implements in the substrate world. Agent gains its agency by simulating the substrate world in its mind and choosing the non-random direction of a favorable future for its own existence (survival of its essential structure). This constant back-and-forth—between physical interaction and representational interpretation—is the heartbeat of an autonomous cognitive system. In what follows, we formalize this interplay via category theory notions, laying the groundwork for the NEOLAF architecture’s learning and reasoning processes.

3.3 The Structure of Representation: The Duality of Knowledge ($\mathcal{K}$) and Experience ($\mathcal{E}$)

Within the vast category of Representation $\mathbb{R}$, we distinguish two structured subcategories that are central to any post-birth learning agent (such as NEOLAF agent):

To delve deeper into the agent’s cognitive processes, we refine the representation category $\mathbb{R}$ into more specific subdivisions that capture different aspects of the agent’s internal state. In particular, we distinguish between the agent’s representation of accumulated **knowledge** and that of its temporal (substrate) **experiences**. Both of these reside in the representational domain ($\mathbb{R}$), but they play different roles and have different structural properties. We will model them as subcategories of $\mathbb{R}$:

- $\mathcal{K}$: the category of *knowledge* (structured, long-lived representations).
- $\mathcal{E}$: the category of *experience* (episodic, transient representations derived from interactions).

Knowledge ($\mathcal{K}$). The knowledge category represents the agent’s internal model of the world and itself. Objects in $\mathcal{K}$ are things like facts, concepts, schemas, and skills that the agent has learned; in the context of NEOLAF, a large portion of the knowledge is stored explicitly in symbolic form (e.g. as KSTAR records, graphs, or knowledge bases) or implicitly in the parameters of neural networks. We can think of each object in $\mathcal{K}$ as an integrated knowledge state or a particular piece of knowledge. For example, one object might be the agent’s knowledge about how a certain tool works, or a generalized concept like “heat causes expansion.” Morphisms in $\mathcal{K}$ represent logical or inferential relationships between knowledge pieces, or the processes of updating and deriving knowledge. For instance, a morphism in $\mathcal{K}$ could be an inference rule that derives a new fact from existing facts, or a refinement relation that updates an old hypothesis into a new one in light of evidence. Because knowledge in $\mathcal{K}$ is structured and typically designed to be consistent and well-organized, morphisms in $\mathcal{K}$ often preserve and reflect that structure (for example, adding a new entry to a knowledge graph or merging two concepts along a taxonomy).

Experience ($\mathcal{E}$). The experience category captures the immediate informational content that the agent gains through interaction with its environment. Objects in $\mathcal{E}$ are episodic events or observations—concrete instances of what the agent has sensed or done and what outcomes were observed. For example, an object in $\mathcal{E}$ might be a particular trial the agent conducted: a specific situation it encountered, the action it took, and the result that followed (essentially a single K–S–T–A–R tuple, in the terminology of the KSTAR framework). Another object could be a raw sensory snapshot at a given moment. Morphisms in $\mathcal{E}$ can be thought of as temporal or causal relationships between experiences (e.g. one experience happening after another, or one experience being the consequence of another). They could also represent transformations from raw observations to interpreted representations (for instance, a morphism that takes a high-dimensional sensor input and yields a classified event or a symbolic description). Experiences in $\mathcal{E}$ tend to be transient and particular: each experience is tied to a specific time and context. However, by mapping experiences into the knowledge category, the agent can abstract and generalize from individual episodes to form enduring knowledge.

By modeling Knowledge and Experience as categories (or at least as structured collections of representations with morphisms between them), we lay a formal foundation for describing learning and memory. In NEOLAF, knowledge and experience interact continuously: knowledge provides the lens through which new experience is interpreted, and new experience provides the data through which knowledge is expanded or corrected. This interplay will be captured by the functorial relationships we define next. It is also mirrored in the architecture’s memory design: knowledge is stored in long-term memory structures (e.g. an explicit symbolic knowledge base, corresponding to $\mathcal{K}$), while experiences are first captured in short-term memory or episodic memory (corresponding to $\mathcal{E}$) before being consolidated.

Knowledge and experience interact continuously: knowledge provides the lens through which new experience is interpreted, and new experience provides the data through which knowledge is expanded or corrected.

3.4 The KSTAR Tuple: An Agent-Centric Knowledge Representation

To make knowledge actionable for an agent, NEOLAF employs a specific, agent-centric representational structure: the KSTAR tuple. An object of knowledge $K \in \mathcal{K}$ is represented as a tuple $(S, T, A, \hat{E})$, where S is the perceived Situation, T is the active Task, A is a proposed Action plan,

and $\hat{E}$ is the Expected Result of that action. This contrasts with more passive knowledge representation paradigms, as it intrinsically links what is known to what can be done. It is a representation of knowledge-for-action.

3.5 The KSTAR Process: Reconciling Prediction with Reality

The core learning process in NEOLAF is a cycle of reconciling prediction with reality. Using its current knowledge, the agent formulates a plan and predicts an **Expected Result** ($\hat{E}$). It then executes this plan in the substrate, observes the outcome, and represents this as an **Actual Result** (R). The discrepancy or alignment between $\hat{E}$ and R is the fundamental signal for learning.

3.6 The Neuro-Symbolic Engine for Learning

The KSTAR process is implemented via a synergistic neuro-symbolic engine.

1. **Plan and Prediction Generation (Neural+Symbolic):** A Transformer-based LLM takes the current context (S, T, K_t) and generates a structured action plan A and a prediction $\hat{E}$. This is an imaginative, heuristic process that leverages the vast patterns learned during pre-training and the subsequent symbolic computation executed by the agent's symbolic facilities such as a plan and prediction generator using code generated by the LLM.
2. **Plan Enactment (Symbolic):** A deterministic symbolic engine (or the environment itself) executes the plan A . This might involve running code, calling an API, or activating a robotic effector.
3. **Observation and Comparison:** The agent perceives the outcome, creating a representation of the Actual Result R . The difference between R and the neurally-generated $\hat{E}$ creates the feedback for the learning loop.

3.7 Formalizing the Learning Composition

We formalize this learning process using two key concepts from category theory.

- **Natural Transformation (η):** The comparison between the predicted outcome and the actual outcome is modeled as a natural transformation. Let F be the predictive functor that maps a context to an expected outcome $\hat{E}$, and G be the observational functor that maps the same context to an actual outcome R . The learning signal—the delta or error—is a natural transformation $\eta : F \Rightarrow G$. For each context x , the component $\eta_x : F(x) \rightarrow G(x)$ is the morphism that represents the "correction" needed to turn the prediction into reality. This η is the formal representation of the learning catalyst.
- **Monadic Multiplication (μ):** To ensure that learning is coherent over time, we model the overall learning cycle as a monad. An endofunctor $\mathbf{T} = \mathbf{L} \circ \mathbf{P}$ on $\mathcal{K}$ represents one full cycle of planning and learning. The monadic multiplication, $\mu : \mathbf{T} \circ \mathbf{T} \Rightarrow \mathbf{T}$, is a natural transformation that "flattens" nested learning cycles. It ensures that learning from two experiences in sequence is consolidated into a single, coherent update to the agent's knowledge base.

3.8 Building Cognitive Functors from Transformers

This entire formal process is made computationally tractable by using the Transformer as the engine for constructing the key cognitive functors.

- The **Plan-Execute functor** ($\mathbf{P} : \mathcal{K} \rightarrow \mathcal{E}$) is implemented by the LLM, which takes a knowledge state K and generates a planned experience $(S, T, A, \hat{E})$.
- The **Learn functor** ($\mathbf{L} : \mathcal{E} \rightarrow \mathcal{K}$) is implemented by the full neuro-symbolic loop that executes the plan, compares the result, and formulates an update to be stored in the knowledge base.

The Transformer’s ability to map between different representations (e.g., natural language descriptions to structured action plans) makes it a practical instantiation of these abstract, structure-preserving maps, thus completing the bridge from metaphysical principles to a working cognitive architecture.

4 The KSTAR Process and NEOLAF Agent Formalism

4.1 Overview of the KSTAR Framework

The **KSTAR** framework—an acronym for *Knowledge, Situation, Task, Action, Result*—provides a structured, multi-layer process by which a **NEOLAF** agent learns from experience and incrementally builds expertise. KSTAR serves as both a representation of the *knowledge–experience duality* and a continual process for acquiring new knowledge through experience. In essence, every interaction or problem-solving episode (an *encounter*) is broken down into four fundamental components (Situation, Task, Action, Result), which together yield new or updated Knowledge for the agent. This cyclical process is depicted schematically in Figure 2. By iterating through KSTAR cycles across diverse encounters, the agent engages in ongoing, recursive learning and adaptation, much like a human accruing wisdom from successive experiences.

Figure 2: Overview of the KSTAR process (Knowledge, Situation, Task, Action, Result) as a cycle. The agent leverages prior Knowledge K to interpret the Situation and Task, plans Actions to achieve the Task, and obtains a Result which is then evaluated and encoded back into Knowledge. This loop illustrates the knowledge–experience duality and continuous learning in NEOLAF agents.

Unlike a purely reactive system, a NEOLAF agent leveraging KSTAR engages in deliberate reasoning and learning at each step. The KSTAR breakdown enforces a logical flow: first understanding the *Situation*, then identifying the *Task*, devising and executing an *Action* plan, and finally evaluating the *Result*. Each component addresses a distinct layer of cognition, yet they operate in concert. Crucially, the agent’s prior **Knowledge** (both what it has explicitly learned from past encounters and what is implicitly encoded in its neural models) influences each step and is in turn updated by the outcome. This interplay between accumulated knowledge and new experience is what we term the “knowledge–experience duality” — knowledge guides actions, and experience grows knowledge.

The KSTAR framework is inspired in part by human problem-solving and learning cycles. It resembles the well-known STAR method (Situation, Task, Action, Result) used in cognitive psychology and even job interview contexts for recounting experiences, but with the explicit addition of the *Knowledge* component that closes the loop. By including Knowledge as both an input and output, KSTAR emphasizes learning: each cycle begins with existing knowledge and ends with refined knowledge. In other words, KSTAR provides a systematic scaffold for a never-ending learning agent, enabling it to break down any encounter into manageable parts and to consolidate the lessons from that encounter into its knowledge base. Table 1 summarizes the role of each component, and we formalize each below.

In the following sections, we provide formal definitions for each component and describe the KSTAR process mathematically. We then discuss how this process is implemented in the NEOLAF agent, which uniquely integrates neural and symbolic capabilities at each layer.

4.2 Formal Definitions and Categorical Formalism of KSTAR

To rigorously define the KSTAR process, we introduce formal notations for each component and describe the relationships between them. We then present a categorical perspective, treating the entire KSTAR cycle as a compositional transformation on the agent’s state (knowledge), which provides a high-level algebraic understanding of sequential learning.

KSTAR Component	Role in the Learning Cycle
Situation (S)	The context or environment of the encounter. Describes the initial state, including relevant environmental factors and agent’s context. Serves as the backdrop that frames what problem needs to be solved.
Task (T)	The specific objective or goal to achieve in the given situation. May be decomposed into sub-tasks or include implicit co-tasks such as planning, forecasting, and grounding. Defines what constitutes success for the encounter.
Action (A)	The plan and sequence of steps executed by the agent to address the task. Encompasses both the formulated plan (expected actions) and the actual execution. Can be represented in structured form (e.g. an agent–skill–constraint triple).
Result (R)	The outcome of executing the actions in the given situation, relative to the task. Includes the agent’s forecasted expected result and the observed actual result. The result is evaluated to determine success or failure and gather feedback.
Knowledge (K)	The agent’s internal knowledge state <i>before</i> and <i>after</i> the encounter. Prior knowledge (including memories and learned parameters) informs Situation analysis, Task planning, and Action selection. After the Result, knowledge is updated by incorporating the new experience (storing it as explicit memory and/or adjusting implicit knowledge).

Table 1: Summary of the KSTAR components and their roles. An encounter is analyzed into S, T, A, R, and yields an update to K.

4.2.1 Situations and Context Representation

Definition: Situation

A **Situation**, denoted S , represents the external context or environment in which an encounter takes place. Formally, we can model a situation as a state s in some state space S , which encapsulates all relevant information about the environment and context at the start of the encounter.

In practical terms, the situation might include the physical environment, the current world state, a problem description, or any contextual data that frames the problem. For example, in a tutoring scenario, the situation could be the text of a math problem and the student’s current knowledge state; in a robotic scenario, the situation might be sensor readings and a description of the environment. The situation provides the conditions under which the agent must operate. The agent’s first step is to *identify or interpret the situation*, which often involves perception or comprehension (e.g. reading a problem statement, observing a scene). We assume the agent has a function $Interpret : S \rightarrow \mathcal{I}$ mapping raw environment data to an internal situation representation i that it can reason about. This internal representation may be symbolic (e.g. a set of facts about the world) and/or subsymbolic (e.g. a vector of features produced by a neural network). Identifying the situation establishes the context for everything that follows.

Example (Situation). Suppose our NEOLAF agent is presented with a new problem: “A farmer has 5 apples and gives away 2. How many apples are left?” The situation S here consists of the textual description of the problem and any relevant context (perhaps the fact that this is an arithmetic word problem, the difficulty level, etc.). The agent’s perception (via its language model) reads and interprets this text, constructing an internal representation of the situation: e.g. a list of facts ‘farmer initially has 5 apples’, ‘farmer gives away 2 apples’ and a question “how many left”.

4.2.2 Tasks and Goal Specification

Definition: Task

A **Task**, denoted T , is the objective or goal that the agent must accomplish within the given situation. Formally, a task can be viewed as a proposition or desired outcome defined on the situation, or more generally an element t from a set $\mathcal{T}$ of possible goals. A task may be associated with a set of success criteria or a utility function $U(t)$ that evaluates outcomes.

The task answers the question: *What is the agent trying to achieve?* It is derived from or given by the situation. In many cases, the task is explicitly provided (e.g. the question asked, the instruction given by a user). In other cases, the agent might have to infer the task from context or higher-level goals. In the KSTAR process, once the situation is understood, the agent defines or identifies the task to be solved. This involves recognizing the objective and possibly breaking it down into subgoals. Formally, we can think of a task as imposing a set of conditions on the result R (for example, “find the number of apples left” means the result should be a numeric answer satisfying the arithmetic).

Tasks can often be *hierarchical* or compound. The KSTAR framework allows for a task to be decomposed into smaller sub-tasks if necessary. Moreover, there are “implied co-tasks” inherent in solving any non-trivial task, notably **planning**, **forecasting**, and **grounding**.

- *Planning* refers to formulating a strategy or sequence of steps to reach the goal.
- *Forecasting* refers to predicting intermediate outcomes or the final result before acting.
- *Grounding* refers to connecting abstract plans to real-world execution and truth-checking the outcomes (e.g. verifying a result against facts or constraints).

These co-tasks are not separate goals themselves but auxiliary cognitive tasks the agent performs as part of working towards the main task. In formal terms, one could treat them as additional tasks $T_{\text{plan}}, T_{\text{forecast}}, T_{\text{ground}}$ that run in parallel to the main task T_{main} , all of which must be satisfied for the encounter to be fully successful. However, for simplicity, we include these within the overall task execution process rather than treating them as separate entries in the KSTAR tuple.

Example (Task). In our arithmetic problem example, the task T is “calculate how many apples are left.” The agent formally represents this as the goal of determining the value of $(5 \text{ apples} - 2 \text{ apples})$. The success criterion is obtaining the correct difference. Planning in this context might be trivial (it’s a single subtraction step), forecasting might involve expecting that the result should be an integer and roughly around 3, and grounding would involve checking that the answer is non-negative and makes sense (it would not make sense to have a negative number of apples in this context).

4.2.3 Actions: Planning and Execution

Definition: Action

An **Action**, denoted A , is the plan or sequence of operations the agent executes in order to accomplish the task T in situation S . We formalize an action as an element a of an action space $\mathcal{A}$, which may consist of composite sequences $a = (a_1; a_2; \dots; a_n)$ representing a plan with multiple steps.

In the KSTAR process, the agent first *plans and then executes* the action(s). Planning transforms the understanding of the situation and task into a concrete approach: the agent decides *what to do* to reach the goal. Execution then carries out that plan in the environment.

We can distinguish between the *planned action sequence* and the *actual action taken*, especially if there is any deviation or if external factors alter execution. The framework can accommodate this by letting A_{plan} represent the intended plan and A_{exec} the actual executed actions. In many cases, these will coincide (the agent executes exactly what it planned), but it’s useful to keep the distinction for completeness.

Formally, we might model a complex action as a sequence of functions or operations that change some state. For example, if the environment state is $s \in \mathcal{S}$, an action a could be interpreted as

a state-transforming function $a : \mathcal{S} \rightarrow \mathcal{S}$ (for an embodied agent) or as an inference step (for a purely cognitive agent). The result of executing a on state s yields a new state $s' = a(s)$. In a purely informational task like solving a math problem, the “action” is a cognitive or computational procedure (e.g. performing a subtraction or a proof).

NEOLAF agents can represent actions in a structured manner. One convenient representation is a triple or tuple capturing the agent, the operation/skill, and any parameters or constraints. For instance, an action might be described by (agent, verb, object) format, or more specifically an **agent–skill–constraint** triple. Here, “agent” refers to the actor (often implicitly the NEOLAF agent itself or a sub-module), “skill” refers to the capability or operation used, and “constraint” refers to any contextual limitation or parameter. This subject–verb–object style structure is analogous to a simple sentence describing what the agent does, which aids interpretability. For example, an action in a tutoring context might be “(Agent, *apply-formula*, *distance=rate*time*)” meaning the agent applies a particular formula given the context that distance equals rate times time.

After planning, the agent executes the actions in the environment. Execution may involve interacting with external tools or calling sub-algorithms. In NEOLAF’s design, the agent can leverage its neural components (e.g. an LLM) to carry out certain computations or its symbolic components to fetch facts or perform logical inferences as part of the action sequence. The sequence of actions and intermediate states might be recorded for analysis or explanation.

Example (Action). In the apple problem, the plan A_{plan} could be a single step: “subtract 2 from 5.” The agent might represent this action in a structured way, e.g. (Agent, *subtract*, 2) with an understanding that it applies to the number 5 from the situation. It then executes this computation. In a more complex scenario, say a multi-step algebra problem, the plan might involve several steps (apply a formula, simplify an expression, etc.), which the agent would list out and then execute in sequence. Each step could be considered a sub-action ($a_1, a_2, \dots$) within the overall action plan A .

4.2.4 Result: Outcome and Evaluation

Definition: Result

A **Result**, denoted R , is the outcome produced by executing the action A in the context of situation S for task T . We formalize a result as an element r in a result space $\mathcal{R}$, which could be a concrete answer, a new environment state, or a qualitative outcome (e.g. success/failure).

The result has two important aspects in KSTAR: the *expected result* that the agent predicted, and the *actual result* that materialized. Before taking action, a well-prepared agent will **forecast** or predict what it expects to happen (this was earlier noted as a co-task of forecasting). This expectation, call it E , is essentially the agent’s hypothesis of the outcome given its current knowledge and the planned action: $E = \text{Predict}(K, S, T, A_{\text{plan}})$. After executing the action, the agent observes the actual outcome R_{actual} . The actual result could be exactly what was expected, better than expected, or worse/different.

The KSTAR process mandates an *evaluation* step: the agent compares R_{actual} with E (expected) and with the task requirements T . It assesses whether the task objectives have been met and analyzes any discrepancies. Formally, we can define a boolean success metric $\text{Success}(R, T)$ that returns true if result R satisfies task T (for example, if the answer is correct), and possibly a more granular *reward* or *score* function $w(R, T)$ that measures how good the result is (this is useful in learning settings, e.g. a reward signal in reinforcement learning).

If E and R_{actual} differ, the agent notes the error $\Delta = (R_{\text{actual}} - E)$ in some appropriate sense (not necessarily numeric—this could be a logical difference or just a boolean indicating success/failure). Additionally, the agent should **ground** the result in reality: check that the result is logically and factually consistent with known constraints or truths. Grounding might involve verifying the result against an external truth source or domain theory. In our formal view, grounding can be seen as a validation function $G(R, S)$ that returns true if R is plausible in situation S (e.g. a physical outcome that respects the laws of physics, or an answer that doesn’t violate known facts).

After evaluation, the outcome R (along with the evaluation information) is ready to be fed back into the agent’s knowledge. The quadruple (S, T, A, R) essentially forms a complete experience data point.

Example (Result). In the apple subtraction example, the agent’s expected result E might be “3 apples remain,” predicted before actually doing the calculation (the agent might anticipate that removing 2 from 5 leaves a small positive number). It then executes the subtraction action and obtains the actual result $R_{\text{actual}} = 3$. It compares this with the expectation — here they match, which indicates success. The agent then checks against the task: indeed, “3” correctly answers the question, so $\text{Success}(R, T) = \text{true}$. Grounding in this simple case might be trivial (it is perfectly plausible to have 3 apples left). If, in a different scenario, the expected and actual results didn’t match (say the agent expected 4 but got 3), the agent would flag the discrepancy and possibly investigate why its expectation was off (e.g. a miscalculation in expectation or an unexpected event during execution).

4.2.5 Knowledge: Dual Memory and Learning Update

Finally, we consider the **Knowledge** component, which both initiates and concludes the KSTAR cycle. The agent’s knowledge K influences how it perceives S , defines T , plans A , and predicts R ; after the encounter, the knowledge is updated to K' by incorporating the new experience.

Definition: Knowledge State

A **Knowledge state**, K , represents the agent’s internal store of information, skills, and learned parameters at a given time. We model the knowledge state as a composite structure $K = (M_{\text{expl}}, M_{\text{impl}})$ consisting of two parts:

- M_{expl} : **Explicit memory**, a symbolic knowledge base of stored facts, rules, and past experiences (often represented in structured forms such as logic formulas, graphs, or KSTAR records).
- M_{impl} : **Implicit memory**, the sub-symbolic knowledge encoded in the parameters of the agent’s learned models (e.g. the weights of an LLM or neural network components).

This dual-memory representation mirrors the NEOLAF agent’s design. The explicit memory is like the agent’s notebook or diary, where each encounter’s details (the situation, task, action, result, and lessons learned) can be logged. Indeed, we can imagine that for each completed KSTAR cycle (encounter) the agent appends an entry to M_{expl} capturing (S, T, A, R) along with meta-information like the date/time, success evaluation, and any extracted “moral” or learned rule from the experience. Over time, M_{expl} becomes a rich episodic memory. This is analogous to case-based reasoning in classical AI or the “experience replay” buffer in reinforcement learning, except that here the entries are structured in the KSTAR format which facilitates search and reuse. The explicit memory allows for *interpretable knowledge*: a human or another agent can inspect it to see what the agent has done and learned in the past (hence enhancing explainability).

Implicit memory, M_{impl} , refers to the information the agent has internalized into its parametric models via learning algorithms. This might be updated in a more gradual, statistical manner. For instance, after accumulating a number of experiences in explicit memory, the agent can invoke a learning procedure to fine-tune its neural network (LLM) on those experiences, thereby embedding that knowledge implicitly. This is akin to a human solidifying knowledge through practice or sleep-mediated consolidation (an analogy explicitly drawn in the NEOLAF framework, where offline fine-tuning is likened to consolidating memory during sleep). The implicit memory gives the agent the ability to generalize and pattern-match beyond the specific cases it has seen; it’s responsible for the agent’s intuitive, fast reasoning (System 1), whereas the explicit memory supports deliberative, logical reasoning (System 2).

Given these, the knowledge update at the end of an encounter can be described formally by a function U :

Definition: Knowledge Update Function

The knowledge update function U takes the prior knowledge and the details of the encounter to produce a new knowledge state: $K' = U(K, S, T, A, R)$. In general, U will update the explicit memory M_{expl} by adding a record of (S, T, A, R) (and any inferences drawn from it), and update the implicit memory M_{impl} by learning from the experience (which may be immediate or deferred).

For example, U might add the pair $(\text{Problem} : 5 - 2, \text{Solution} : 3)$ to the agent’s knowledge base as a new fact or reinforce the rule “subtracting 2 from 5 gives 3.” If the agent made an error or had a wrong expectation, U would incorporate the correction: e.g. “I expected 4 but the result was 3, therefore adjust my mental model or note the exception.” Over time, repeated application of U across many experiences yields an increasingly knowledgeable agent.

It is worth noting that U can be decomposed into two parts corresponding to the two memory types: $M'_{\text{expl}} = U_{\text{expl}}(M_{\text{expl}}, S, T, A, R)$, $M'_{\text{impl}} = U_{\text{impl}}(M_{\text{impl}}, S, T, A, R)$. Often U_{expl} is executed after every encounter (online update: e.g. simply logging the case and any immediate conclusions), whereas U_{impl} might be executed less frequently or in batch (offline update: e.g. fine-tuning the neural model on accumulated data overnight). This two-tier update scheme aligns with the fast-slow cognitive duality: immediate explicit learning vs. consolidated implicit learning.

Example (Knowledge Update). Continuing our example, after solving the apple subtraction problem, the agent’s explicit memory M_{expl} is updated with an entry: `Situation: farmer had 5 apples, gave 2 away; Task: find apples left; Action: subtract 2 from 5; Result: 3`. The agent might also generalize a bit of knowledge such as “giving away means subtraction” or reinforce its arithmetic knowledge. If this was the first time the agent encountered such a problem, it has now learned a pattern. If it had a neural component that wasn’t confident about subtraction before, the agent could later fine-tune that component on this new data point (implicit update), so that next time it sees a similar problem, it is more adept. This demonstrates how the agent’s knowledge grows: symbolically, it has a new case in memory (which it can explicitly recall or explain in the future), and subsymbolically, it has a tiny training example to improve its network’s weights.

4.2.6 KSTAR as a Compositional Process (Categorical View)

From a higher-level mathematical standpoint, the entire KSTAR cycle can be viewed as a **state transformation** or arrow that takes an initial knowledge state and produces an updated knowledge state (along with an observable result). This perspective allows us to reason about sequences of encounters abstractly and ensure that learning composes correctly over time. We now outline a categorical formalization of this idea.

Consider the set of all possible knowledge states as $Ob(\mathcal{K})$, the objects of a category $\mathcal{K}$. Each object is a state $K = (M_{\text{expl}}, M_{\text{impl}})$ capturing the agent’s knowledge at a point in time. We define a morphism in this category to represent the process of undergoing a particular encounter (with specific S, T, A, R) that transforms knowledge state K into $K' = U(K, S, T, A, R)$. Let us denote an encounter by its situation-task pair and outcome, e.g. $\langle S, T, R \rangle$ (the action A is implicit in how we get from S, T to R). Then we can write a morphism: $f_{\langle S, T, R \rangle} : K \rightarrow K'$, meaning “given initial knowledge K , experiencing situation S with task T (and obtaining result R) produces a new knowledge state K' .” For brevity, we might label this morphism just by the encounter context, say $f_e : K \rightarrow K'$. The KSTAR process provides the internal structure of this morphism: it factors into sub-steps for Situation, Task, Action, Result, but from the outside it is a single transformation.

Composition: If the agent immediately proceeds to a second encounter $e_2 : \langle S_2, T_2, R_2 \rangle$ after the first, with K' as its starting knowledge, that yields another morphism $f_{e_2} : K' \rightarrow K''$. The sequential nature of learning implies that the overall effect from K to K'' is the composition $f_{e_2} \circ f_{e_1} : K \rightarrow K''$. This composition law holds in general: a sequence of encounters $e_1, e_2, \dots, e_n$ corresponds to a composition of the respective morphisms $K_0 \xrightarrow{f_{e_1}} K_1 \xrightarrow{f_{e_2}} K_2 \rightarrow \dots \rightarrow K_n$. The category $\mathcal{K}$ of knowledge states with these encounter morphisms is thus well-defined. The identity morphism in this category is simply an encounter that changes nothing (perhaps a trivial encounter with no task or one where the agent already knew everything and confirmed it, leaving knowledge unchanged).

This categorical model is useful because it highlights a few important properties:

- Learning is *order-dependent* but associative: $f_{e_3} \circ (f_{e_2} \circ f_{e_1}) = (f_{e_3} \circ f_{e_2}) \circ f_{e_1}$. In other words, if encounter e_1 is followed by e_2 , then e_3 , the final knowledge K_3 is independent of how we parenthesize the sequence—consistent with standard composition in categories. Practically, this means the end state after three experiences is the same no matter how we conceptually group the experiences.
- There is a notion of *cancellativity* only in a very limited sense: one generally cannot “undo” an experience (short of explicitly forgetting). So most morphisms f_e will not be invertible. The category $\mathcal{K}$ is thus a directed acyclic graph of states (if we assume knowledge strictly accumulates or at least doesn’t cycle back to a previous state spontaneously). Each new experience adds (or updates) knowledge, pushing the state forward in a partial order (we might say $K' \succeq K$ if K' has greater or equal knowledge than K).
- The existence of an identity morphism corresponds to the agent doing nothing or experiencing nothing (knowledge remains the same). This is conceptually the “do-nothing” action with no task.

We can further enrich this formalism by noting that each encounter morphism $f_{\langle S, T, R \rangle}$ can be associated with the data of that encounter. In category theory terms, one might consider a functor from a category of *encounters* (where an encounter is an object that includes S, T, A, R data) to this category $\mathcal{K}$ of knowledge states. However, a simpler view is to treat the encounter data as a label on the morphism.

Another perspective is to view the KSTAR cycle as an **endofunctor** or a **monadic structure** on the state space of knowledge. In functional programming terms, this is analogous to a state monad where the state (knowledge) is threaded through computations. Each encounter is like applying a state transition function $K \mapsto K'$ (with some side output R). The monadic bind (composition) just corresponds to sequentially applying state transformations. This aligns with how we model composition above. The KSTAR process thus forms a kind of *learning monad*: the agent’s knowledge state is the internal state carried through computations, and each encounter provides a transformation of that state along with an observable result (which could be considered the monad’s output or effect). Such a view might allow us to apply algebraic reasoning to sequences of learning events and ensure properties like consistency or convergence as the agent goes through many KSTAR cycles.

In summary, the categorical formalism underscores that the KSTAR process is not just a heuristic sequence of steps but a well-defined transformation on the agent’s cognitive state that composes over time. It provides a language to talk about the agent’s learning trajectories: for instance, one can ask whether two different sequences of training experiences lead to isomorphic knowledge states, or whether there exists a shorter path (sequence of encounters) to achieve a certain knowledge level (this relates to curriculum learning, for example). While we will not delve further into category-theoretic properties here, this formal view assures us that the framework has an inherent consistency and mathematical elegance supporting the intuitive process description.

4.2.7 Pseudocode for the KSTAR Cycle

To ground the formalism in a more concrete algorithmic description, Figure 1 provides pseudocode for a single KSTAR cycle as executed by a NEOLAF agent. This illustrates how the components come together in practice within the agent’s control loop.

Each function in the pseudocode corresponds to one or more of the steps we’ve described. For instance, INTERPRETSITUATION might involve an LLM analyzing a text description (neural processing) and extracting key facts (symbolic output). IDENTIFYTASK might look for a question or objective in the interpreted situation (perhaps using prompt-based cues or pattern matching). FORECASTOUTCOME uses the agent’s knowledge to set an expectation; this could be done by simulating the plan mentally or retrieving similar past cases from memory to guess the likely result. PLANACTIONS could involve chain-of-thought reasoning, where the agent (via the LLM or a symbolic planner) lays out a step-by-step solution or action strategy. EXECUTEACTIONS interfaces with the environment or problem-solving API (for a math problem, this might just be performing calculations or invoking an external calculator tool). EVALUATERESULT implements the grounding and comparison between expected and actual, possibly logging any errors or surprises. Finally, UPDATEKNOWLEDGE implements the

Algorithm 1 KSTAR Cycle for a NEOLAF Agent (one encounter)

Require: K (current knowledge state); S (situation input)

Ensure: Updated knowledge state K' after processing the encounter

- 1: $i \leftarrow \text{INTERPRETSITUATION}(S, K)$ $\triangleright$ Parse situation S into internal representation, may use context from K
 - 2: $T \leftarrow \text{IDENTIFYTASK}(i, K)$ $\triangleright$ Determine the goal/task from the situation context
 - 3: $E \leftarrow \text{FORECASTOUTCOME}(K, i, T)$ $\triangleright$ Predict expected result based on current knowledge
 - 4: $A \leftarrow \text{PLAN ACTIONS}(K, i, T, E)$ $\triangleright$ Formulate plan (action sequence) to achieve T
 - 5: $R \leftarrow \text{EXECUTE ACTIONS}(A, S)$ $\triangleright$ Carry out actions in the real situation S , observe actual result R
 - 6: $\text{EVALUATE RESULT}(R, E, T, S)$ $\triangleright$ Compare actual vs expected result, check success criteria
 - 7: $K' \leftarrow \text{UPDATE KNOWLEDGE}(K, S, T, A, E, R)$ $\triangleright$ Incorporate this new experience into explicit & implicit memory (dual update)
 - 8: **return** K'
-

U function, updating both the explicit memory (storing this encounter, perhaps with a tag like “solved correctly” or “made mistake in step 2”) and scheduling an implicit memory update (maybe flagging this problem for inclusion in the next model fine-tuning batch).

This pseudocode abstractly captures one cycle. In a continuous learning setting, the agent would loop over this as new situations arise, always carrying its updated K forward. In a multitasking scenario, the agent might interleave multiple such cycles (hence the earlier description of KSTAR as “multitasking”, meaning an agent can manage concurrent or interleaved tasks, though for simplicity we describe one at a time here).

4.3 Neural-Symbolic Integration in the KSTAR Process

A key motivation for the NEOLAF cognitive architecture is to **mix and match neural and symbolic capabilities** in a unified agent framework, capitalizing on the strengths of each. The KSTAR process is specifically designed to facilitate this synergy: each layer (Situation, Task, Action, Result, and the overarching Knowledge management) involves subtasks that are naturally suited to either neural approaches, symbolic approaches, or a collaboration of both. In this section, we analyze how neural and symbolic computation contribute to each phase of KSTAR, and why a dual (hybrid) system is essential for the agent’s performance and continual learning.

4.3.1 System 1 and System 2 Dynamics Across KSTAR Layers

Psychologically, our design is informed by dual-process theory: **System 1** (fast, intuitive, pattern-based reasoning) and **System 2** (slow, deliberate, logical reasoning). In NEOLAF:

- The *neural components* (especially the large language model and any neural perception modules) correspond to System 1: they excel at rapid pattern recognition, natural language understanding, and generating plausible continuations or ideas. They operate over learned implicit knowledge.
- The *symbolic components* (the explicit memory, logic-based planners, knowledge graphs, external tools, etc.) correspond to System 2: they handle explicit reasoning, precise computation, planning, and enforce consistency with stored knowledge or rules.

During a KSTAR cycle, these two “systems” interact closely. We can map the involvement of each as follows:

- **Situation interpretation (S):** Often initiated by neural processing, especially if the situation is described in unstructured data (text, images). For example, an LLM is used to parse a textual problem or a vision model to parse an image scene. The output of this can be a symbolic representation (facts extracted) that populates the explicit context. Symbolic knowledge may also be queried: e.g. recalling from explicit memory if this situation is

similar to a past one, or looking up relevant facts (like “apples are countable objects”). Neural => produce representation; Symbolic => retrieve relevant prior cases or facts.

-
- **Task identification (T):** If the task is explicitly stated in language, the neural model helps interpret it (e.g. understanding a question). If the task needs inferring, the neural model’s intuition might guess the implicit goal (as humans use intuition to figure out what to do in novel scenarios). Symbolic logic might then formalize the task: for instance, translating a natural language instruction into a formal goal representation or a set of constraints. This formal task representation can be stored or reasoned about more rigorously. Thus neural helps with comprehension; symbolic helps with formalization.
-
- **Action planning (A):** This is a critical juncture for collaboration. Neural approaches (like chain-of-thought prompting in an LLM) can brainstorm possible steps or even generate a draft plan in natural language. Because neural networks leverage learned patterns, the LLM might quickly recall a similar problem solution and suggest a strategy. However, purely neural plans might sometimes be flawed or not fully logically consistent (LLMs can produce incorrect reasoning steps). The symbolic planner or reasoning module can take these suggestions and verify or refine them. For example, a symbolic math solver could verify each step of a solution outline produced by the LLM, or a rule-based system could prune nonsensical steps. Moreover, symbolic components can ensure constraints are met (like physical laws, or that a plan doesn’t violate known rules in the knowledge base). In execution, if the plan involves calculation or querying a database, symbolic tools (like a calculator, a theorem prover, or an API call) are engaged. The neural component might be used again within the action execution for tasks like generating text (e.g. writing an explanation, if the task were to explain something) or making perceptual judgments.
- **Result forecasting (part of R):** The agent’s ability to predict the outcome before it happens relies on internal simulation. A neural model might simulate “imagination” of what could happen (for instance, an LLM can be prompted: “What do I expect the answer to be?” or in a robotics context, a learned dynamics model predicts the result of an action). Symbolic knowledge also contributes: if the agent has a physics engine or a deterministic rule, it can calculate a predicted outcome. In many cases, a combination is used: the agent might use known rules for aspects it is confident (symbolic), and use the neural predictor for uncertain parts. This yields the expected result E .
- **Result evaluation (R continued):** Here, symbolic logic plays a primary role in checking correctness: comparing R_{actual} with T ’s success criteria is often a logical or arithmetic check. If a formal ground truth is available (e.g. the correct answer to the math problem, or a specification in the task), the agent’s symbolic component can directly verify if the result matches. Neural components might be less needed for checking (though in tasks without a clear ground truth, an LLM might be asked to judge qualitatively if the outcome seems good – e.g. in a creative task, the LLM might self-evaluate if a story is coherent, though even that can be aided by rules or checklists). Grounding the result in reality is also often symbolic: for instance, confirming that a plan’s result didn’t violate any constraints involves checking against the knowledge base or external truth sources.
- **Knowledge update (K):** Storing the experience (explicit memory) is inherently symbolic – we want a structured record. The agent likely converts the outcome and key lessons into a structured form. This might involve symbolic generalization: for example, inferring a general rule or extracting a moral (“if subtracting leads to negative, maybe the task was impossible” or “if a strategy failed, try a different one next time”). Symbolic induction methods can be used here (learning rules from examples). On the other hand, improving implicit knowledge is done via neural learning algorithms: adjusting weights, embedding new facts. This might require neural network training procedures, which are numeric in nature (stochastic gradient descent on a loss function, for example). The decision of what data to train on or what part of the model to tune might itself be guided by symbolic logic (e.g. the agent might decide “I got three algebra problems wrong of type X; create a training mini-batch of those for the neural network to practice on”). So symbolic decides how to curate and schedule learning, while neural does the heavy lifting of absorbing patterns from

data. This complementary process ensures the agent improves both its explicit repertoire of knowledge and its implicit efficiency and intuition.

To illustrate the need for both types of reasoning, consider a scenario that exposes the limitations of purely one approach:

- A purely symbolic agent (without neural intuition) might have perfect logical rigor but could struggle with interpreting rich, ambiguous inputs or discovering new strategies without extensive human-provided knowledge. For instance, reading a complicated paragraph to identify the task is non-trivial to hard-code; an LLM’s learned language ability is extremely useful here.
- A purely neural agent (without symbolic memory or logic) might process language well and even solve many problems by pattern recognition, but it would have no explicit record of what it learned from each problem. It might repeat mistakes because it lacks an explicit mechanism to compare expected vs actual outcomes in a principled way. Moreover, explaining its reasoning or ensuring consistency over long sequences is problematic. It might also be sample-inefficient: without storing explicit facts, it has to relearn patterns again and again through weights.

NEOLAF’s dual architecture addresses these issues. The explicit KSTAR log provides *explainability* and a form of long-term memory that can be queried (addressing the “black box” nature of neural nets). The implicit learning provides *generalization* and efficiency, so the agent doesn’t need a human to pre-program every rule (addressing the brittleness of purely symbolic systems). By collaborating, the neural and symbolic parts handle tasks neither could easily do alone. This collaborative interplay is orchestrated via the KSTAR framework: each step is an opportunity for one or both paradigms to contribute.

4.3.2 Causal Process Mapping and Qualitative Analysis

Figure 3 illustrates how information flows between the neural and symbolic components during the KSTAR cycle, highlighting the causal dependencies. At a high level, we can identify a causal chain: *Situation* $\rightarrow$ *Task* $\rightarrow$ *Action* $\rightarrow$ *Result* $\rightarrow$ (back to) *Knowledge*. Each arrow in this chain is influenced by either neural or symbolic processing (or both), and any breakdown in one stage can propagate causally to affect later stages and the ultimate knowledge update.

Figure 3: Neural-symbolic dual memory architecture in the NEOLAF agent. The agent has an implicit (neural) memory component (e.g. an LLM with learned weights) enabling fast pattern recognition and intuitive reasoning (System 1), and an explicit (symbolic) memory (e.g. a knowledge base of structured KSTAR records) enabling logical reasoning, planning, and explanation (System 2). Both types of memory interact during the KSTAR cycle: the neural component provides predictions and interpretations, while the symbolic component provides reliable recall, constraint checking, and long-term consolidation of experiences.

Qualitatively, we can analyze a single encounter as follows:

- The agent’s prior knowledge (*K*) causally affects how it perceives the situation and what task it identifies. If *K* lacks relevant knowledge or has misconceptions, that will cause an incorrect or incomplete understanding of *S* or *T*. For example, if the agent didn’t know what “gives away” means in the apple problem, it might misidentify the task. This shows a cause-effect: knowledge gaps cause misinterpretation.
- The situation and task together cause the agent to formulate certain actions. A misunderstanding in *S* or *T* will lead to a flawed plan *A*. Conversely, if those are correct, but the agent’s planning ability (part of knowledge/skills) is weak, it might still choose a suboptimal action. For instance, if the task is to solve a puzzle, and the agent’s planning (perhaps via neural chain-of-thought) goes astray at step 3, the resulting action sequence will be faulty. Thus, we see cause flowing from earlier stages: *S* and *T* cause *A*.
- The actions cause the result *R* in the environment. If the agent performed the right moves, the result likely achieves the task; if it acted incorrectly, the result may be failure or

an unexpected outcome. External stochasticity or environmental complexity also mean A doesn't deterministically guarantee R —there can be external causes for R (noise, adversaries, etc.), but from the agent's perspective we treat R as caused by its A under S .

- The comparison between expected and actual result, and the success or failure with respect to T , causes the agent to update its knowledge K' . A large deviation or a failure is a strong signal to learn (e.g. "I was very wrong, so I clearly lack some knowledge or had a false assumption"). A perfect success might still cause learning (the agent reinforces that its approach works, or stores the example for future reference). Minor differences might cause a small adjustment. The causal link here is essentially the classical learning feedback loop: error or reward signals drive knowledge change.

By mapping these causal relations, we can create a process flow diagram (as conceptually shown in Figure 2 and detailed in Figure 3) that helps diagnose where things can go wrong and how improvements in one part affect the whole:

- If the agent's neural component misinterprets the situation, the symbolic planner might be operating on wrong premises, yielding a chain of errors. In response, the system can be designed to have checks (e.g. after interpretation, query the explicit memory or an ontology: "does this situation make sense?").
- If the agent plans poorly (maybe the neural chain-of-thought hallucinated a step), the explicit knowledge or a verification module can catch it (for instance, verifying the plan logically before execution). If not caught, a failed result will eventually lead to knowledge update where the agent can learn that the plan was flawed.
- If the result is unexpected, the agent's knowledge update might attempt to find the cause: was it a lack of knowledge? Or was it random chance? This introspection can itself be a symbolic causal analysis. NEOLAF could employ a technique where after a surprising failure, the agent explicitly analyzes the causal chain: "I assumed X at step 2, which turned out false, leading to failure. Therefore, I learn X is false in this context." Such causal learning is powerful: it means the agent doesn't just memorize the outcome but understands *why* it happened. Implementing this requires symbolic reasoning (causal graphs, dependency tracing) possibly augmented by neural help (the LLM might assist by generating explanations which the symbolic part can refine and store).

To make this concrete, let's qualitatively examine an example with potential pitfalls:

Extended Example. Imagine a NEOLAF agent in a science lab domain. Situation: the agent is given a task to determine why a plant is wilting. The context includes: the plant hasn't been watered for 5 days, and it's been kept in a dark room. Task: diagnose the cause of wilting. The agent's knowledge includes basic botany (plants need water and light). The agent identifies the situation and task correctly (thanks to neural NLP understanding of the problem description). Now, the agent plans actions: perhaps query its knowledge base for causes of wilting, then perform an experiment or ask a follow-up question. Suppose the neural component overenthusiastically generates a plan: "Maybe the soil has a disease; test the soil." The symbolic component, checking its knowledge base, finds a contradiction: lack of water is a known primary cause, which hasn't been addressed yet. The symbolic reasoning might then adjust the plan to first check soil moisture or watering schedule. The agent executes the plan: finds the soil is dry. Result: plant is wilting due to drought. Expected vs actual: the agent expected drought to be the issue (if the symbolic check influenced its expectation), and indeed confirmed it. Knowledge update: the agent logs this case and perhaps strengthens the rule "if plant is wilting and no water in days, then cause is drought."

Now consider if the agent lacked the knowledge that plants need water (a knowledge gap in K). The neural component might not mention water at all and focus on exotic causes like disease (pattern-matching something else). The result would be the agent possibly performing unnecessary tests, not immediately solving the task. Eventually, if an external feedback (say a human says "It just needed water!") provides the result, the agent updates its knowledge, learning the importance of watering. This new knowledge K' will causally ensure that in future similar situations, the agent first considers water. This example shows how a missing piece in explicit knowledge leads to a neural guess (which can be off-track), and how the result feedback corrects it, adding that piece explicitly for next time.

Thus, the neural-symbolic agent not only performs tasks but incrementally builds a world model capturing causal relations (like watering causes plant health) from experience. KSTAR explicitly facilitates this by requiring the agent to articulate situation, task, action, result, and thus think about the connections between them.

4.3.3 Why Both Neural and Symbolic Capabilities are Required

From the above analyses, it should be clear that neither neural nor symbolic reasoning alone would suffice to implement the full KSTAR process effectively:

- Neural-only agents (e.g. end-to-end deep learning without symbolic memory) would struggle with long-term, explicit learning. They might achieve task success in the moment by pattern matching but forget the instance or fail to abstract the principle. By contrast, KSTAR’s explicit recording ensures lessons are retained. Also, neural models alone have difficulty with reliable planning for complex, multi-step tasks (they can lose coherence in lengthy reasoning). The symbolic planning compensates for that.
- Symbolic-only agents (e.g. classical AI planners without learning) lack the adaptability and perception prowess. They require all rules upfront and can’t easily refine their own rules without human input. KSTAR’s integration of neural learning allows the agent to autonomously improve by digesting experiences into model parameters, expanding what it can handle (incremental learning).

The NEOLAF architecture, via KSTAR, blends these to yield emergent benefits:

- *Explainability*: Because each encounter is explicitly represented, one can query the agent “why did you do X?” and it can point to its KSTAR record (Situation X, Task Y, I did Action Z because... and Result was...). Pure neural networks would have difficulty pointing to a specific past case influencing a decision, but a NEOLAF agent can, since it literally stores those cases.
- *Continual and multitask learning*: The KSTAR framework lets an agent handle multiple tasks in an ongoing manner, switching contexts and accumulating knowledge without catastrophic forgetting. The explicit memory provides context-specific recall (so knowledge is indexed by situations/tasks), while implicit memory provides generalization across tasks. The symbolic layer can juggle multiple goals and memory entries, something neural networks alone might entangle. Indeed, KSTAR was designed for “ongoing, iterative, and multitasking procedures”.
- *Human-in-the-loop and collaboration*: Since tasks and results are explicitly represented, a human or another agent can inject advice or corrections at the symbolic level (e.g., after the agent’s result, a human teacher could add a note to the knowledge base explaining a mistake). The neural model can then eventually be retrained on that corrected data. This synergy is harder in a purely neural system where knowledge isn’t disentangled and accessible.
- *Efficiency and focus*: The symbolic system can guide the neural system to focus on relevant information. For example, if the situation is very complex with lots of data, the agent’s explicit knowledge might tell it which part is relevant (pruning the search space), rather than letting the neural net drown in possibilities. Conversely, the neural system can handle the “intuitive heavy-lifting” in novel situations where symbolic rules are not yet learned.

Research and early experiments with NEOLAF agents validate these advantages. In a math problem-solving context, a NEOLAF agent employing the KSTAR process was able to incrementally improve its performance on challenging competition problems and even outperformed a baseline LLM (ChatGPT) after sufficient learning. For instance, the agent could solve certain AIME geometry problems by planning a structured solution (Symbolic) while using its neural component to recognize which formula might apply. After solving or failing, it stored the approach, and over time it accumulated strategies for different problem types. This led to demonstrably “superior learning capability” compared to a static model. The dual memory was crucial here: when confronted with a new problem, the agent could recall a similar past problem from explicit memory (even if the neural model didn’t make the connection initially) and adapt that solution, a capability pure neural models lack.

In conclusion, the four layers of the KSTAR process form a comprehensive loop that an intelligent agent must traverse for each learning episode. By embedding both neural and symbolic methods into these layers, the NEOLAF agent ensures that it can interpret complex inputs, rigorously reason through tasks, interact effectively with the environment, and accumulate knowledge over time. The categorical and formal descriptions of KSTAR guarantee that this process is well-defined and composable, giving a solid theoretical foundation for building ever more complex cognitive agents. The qualitative and causal analysis shows that KSTAR is not just an abstract loop, but one that maps onto real cause-effect phenomena in learning and problem-solving. Therefore, the KSTAR framework, implemented through a neural-symbolic architecture, is a powerful approach to achieving continual, explainable, and robust learning in AI agents.

Going forward, we will leverage this framework to design curricula for the agent (sequences of tasks to accelerate learning) and to enable multiple agents to share knowledge within the KSTAR paradigm (as the NEOLAF environment envisions agent societies). By ensuring every experience is decomposed and assimilated through KSTAR, we can have confidence that our agents are not only performing tasks but truly *learning how to learn* in a scalable, interpretable way, combining the best of connectionist and symbolic AI.

Part III

Architecture and Process: How NEOLAF Works

5 The Anatomy of a NEOLAF Agent

Having established the theoretical and formal foundations, we now turn to the concrete structure of a NEOLAF agent. This chapter details the static components of the architecture—what it is made of—and how these components embody the principles of continuous learning, hierarchical memory, and neuro-symbolic synergy.

5.1 Foundational Principles

The NEOLAF architecture is built upon several key principles that guide its design and operation:

- **Continuous Learning:** The agent is not a static entity. It is designed to learn from every interaction, continuously updating its knowledge and improving its performance over time.
- **Hierarchical Memory:** The agent employs a multi-layered memory system to store and process different kinds of knowledge at different levels of abstraction and persistence, mirroring aspects of human cognition.
- **Neuro-Symbolic Synergy:** The architecture fundamentally integrates neural and symbolic computation. It leverages neural networks for pattern recognition and intuitive planning, and symbolic systems for structured reasoning, verification, and explicit knowledge storage.
- **Substrate-Representation Cycle:** The agent’s core function is to execute a perpetual cycle of projecting internal, abstract representations onto the physical substrate through action, and then updating those representations based on perceived outcomes.

5.2 A Layered Architectural Model

To achieve its goals, NEOLAF is conceptualized as a layered system, where each layer performs a distinct function in the cognitive process:

- **The Semantic Engine (LLM):** At the core is a Large Language Model that acts as the primary reasoning and planning engine. It interprets unstructured input, retrieves relevant knowledge, generates hypotheses, and formulates structured action plans.

- **The Qualitative Representation (DSL):** This layer provides a structured "lingua franca" for the agent's internal thought processes. The Semantic Engine translates its reasoning into this explicit symbolic format in agent short-term memory (e.g., Natural language description of observation, causal reasoning, decision; JSON object describing the action status and environmental state; PDDL for detailed plan; Python code for symbolic execution; XAPI statement for capturing experience and results, etc.), which can include or convert to the KSTAR tuple.
- **The Execution Layer (Tools & Environment):** This layer consists of the tools, APIs, and sub-agents that can execute the structured commands generated by the Semantic Engine. This is where the agent interacts with the Substrate, whether by running code, querying a database, or controlling a physical effector.
- **The Symbolic Memory (M2_LTM):** This is the agent's explicit, long-term knowledge base, where validated KSTAR episodes are stored for future retrieval, reasoning, and learning.

5.3 The Cognitive Substrate: A Hierarchical Memory System

The agent's knowledge and operational states are distributed across a hierarchical memory system, designed to support its different modes of thinking (Systems 0, 1, and 2).

5.3.1 M0: Innate Memory (System 0)

M0 constitutes the agent's "Operating System" or "DNA." It is the immutable, pre-written code and foundational architecture residing on the host computer. M0 is not learned but is part of the agent's initial endowment. Its functions include:

- The core executable code for the neural (LLM) and symbolic engines.
- The bootstrapped logic for the KSTAR learning cycle itself.
- The fundamental algorithms for memory access, tool execution, and inter-process communication.

M0 provides the stable substrate upon which all higher-level learning and cognition are built.

5.3.2 M1: Neural Memory (System 1)

M1 is the domain of the neural engine and supports fast, intuitive thinking. It is composed of two parts:

- **M1_LTM (Neural Long-Term Memory):** This corresponds to the learned parameters (weights) of the LLM. It is shaped by initial pre-training and is subsequently updated via periodic fine-tuning ("sleeping cycles") based on the agent's accumulated experiences. M1_LTM represents the agent's internalized skills, implicit knowledge, and learned intuitions.
- **M1_STM (Neural Short-Term Memory):** This is the LLM's active context window at inference time. It is a volatile memory that holds the neural representation of the current prompt, including user input and any information retrieved from long-term memory via Retrieval Augmented Generation (RAG).

5.3.3 M2: Symbolic Memory (System 2)

M2 is the domain of the symbolic engine and supports slow, deliberate, and structured thought. It is also composed of two parts:

- **M2_STM (Symbolic Short-Term Memory):** This is the agent's structured "mental scratchpad." It is a volatile workspace that holds the explicit KSTAR tuple for the current cycle: the perceived *Situation*, the defined *Task*, the generated *Action Plan*, and the observed *Result*. This is where the agent orchestrates and tracks the steps of a deliberate plan.
- **M2_LTM (Symbolic Long-Term Memory):** This is the agent's persistent, structured knowledge base. It stores validated KSTAR episodes, capturing the essence of past experiences, successful (and unsuccessful) plans, and the learning derived from comparing

expectations with reality. This explicit memory is crucial for explainability and future reasoning.

5.4 The Knowledge Transformation Bridge

The power of the NEOLAF architecture lies in the continuous, synergistic interaction between its memory systems, facilitated by a "Knowledge Transformation Bridge." Knowledge does not remain siloed. The primary flow is cyclical:

1. **Grounding Neural Plans (M2_LTM $\rightarrow$ M1_STM):** When forming a plan, the agent retrieves relevant, structured experiences from its symbolic long-term memory (M2_LTM) using Retrieval Augmented Generation (RAG). This explicit knowledge is injected into the neural short-term memory (M1_STM), providing grounding and context for the LLM’s reasoning, leading to more accurate and relevant plans.
2. **Internalizing Experience (M2_LTM $\rightarrow$ M1_LTM):** Periodically, during "sleeping cycles," the validated, symbolic knowledge accumulated in M2_LTM is used as a dataset to fine-tune the LLM. This process distills explicit experiential lessons into the agent’s implicit, intuitive neural long-term memory (M1_LTM), enhancing its core capabilities over time.

This bridge ensures that immediate actions are informed by past explicit experiences, and that those explicit experiences are gradually internalized to build better intuition.

6 The Physiology of Learning: A Walkthrough of an Experiential Cycle

To make the abstract architecture and formalisms concrete, this section details the dynamic process of a single experiential learning cycle. We will use a running example—a mathematical reasoning task—to trace how the agent observes, plans, acts, and learns, illustrating the interplay between its neural and symbolic components and the transformation of information through the memory hierarchy.

Let us consider the following AIME-style problem as our running example: “Let a sequence be defined by $x_1 = 1$ and $x_{m+1} = x_m + 2m$ for all $m \geq 1$. Find x_{100} .”

6.1 Conceptual Overview: The KSTAR Cycle

First, we describe the process at a high, conceptual level, introducing the KSTAR components and the cognitive functors they represent.

- **Situation (S):** The agent perceives the problem. The *Situation* is the full context: “I am an AI agent presented with the task to solve the problem: ‘ $x_1 = 1$, $x_{m+1} = x_m + 2m$, find x_{100} .’”
- **Task (T):** The agent’s control logic (M0) extracts the explicit goal from the situation. The *Task* is: “Calculate the integer value of x_{100} .”
- **Action Plan (A) & Expected Result ($\hat{E}$):** This is the planning phase, an instantiation of the **Plan-Execute functor (P)**. The agent, drawing on its knowledge (M1/M2), formulates a hypothesis. A plausible *Action Plan* would be: “1. Calculate the first few terms to find a pattern. 2. Formulate a closed-form expression for x_m . 3. Use the expression to calculate x_{100} .” The agent’s *Expected Result* ($\hat{E}$) is: “I should be able to get the right answer by executing the plan.”
- **Result (R) & Learning:** This is the enactment and learning phase, an instantiation of the **Learn functor (L)**. The agent executes the plan. The *Actual Result* (R) is the value it computes, e.g., “9901”. The agent then compares R to $\hat{E}$. In this case, the form matches (an integer), but the agent may use an Oracle or internal validation to confirm correctness. The entire episode, including the successful plan, is then encapsulated. The learning catalyst (η), the comparison, drives the update to M2_LTM, reinforcing that this strategy is effective for this class of problem.

6.2 A Detailed Execution Trace

Now, we trace the same example through the specific morphisms and memory systems of the NEOLAF architecture.

6.2.1 Step 1: System 1 Planning (The f_{nn} Morphism)

The process begins in System 1. The problem description is loaded into the LLM’s context window (M1_STM). The control logic (M0) prompts the Semantic Engine (LLM) to generate a solution plan. The LLM, acting as the f_{nn} functor, uses its pre-trained knowledge (M1_LTM) to recognize the problem as a recurrence relation. It might also perform a quick search of its symbolic memory (RAG from M2_LTM) for similar problems. It generates a structured, executable plan and places it into the symbolic workspace (M2_STM).

M2_STM State (Action Plan):

```
plan:
- step: 1
  tool: python_executor
  code: |
    x = {1: 1}
    for m in range(1, 5):
      x[m+1] = x[m] + 2*m
    print(f"First few terms: {x}")
- step: 2
  tool: self_reason
  prompt: "Analyze the sequence x_m and find a closed-form expression."
- step: 3
  tool: python_executor
  code: |
    m = 100
    # Assuming closed_form is m*(m-1)+1
    result = m*(m-1) + 1
    print(result)
```

M2_STM State (Expected Result $\hat{E}$): An integer value.

6.2.2 Step 2: System 2 Execution (The f_{sym} Morphism)

The agent’s symbolic control unit (M0) now acts as the f_{sym} functor, executing the plan from M2_STM step-by-step.

- It executes Step 1. The Python tool returns: “First few terms: 1: 1, 2: 3, 3: 7, 4: 13, 5: 21”. This output is stored back in M2_STM.
- It executes Step 2. The reasoning prompt is sent to the LLM. The LLM analyzes the sequence (1, 3, 7, 13, 21) and might hypothesize the closed form is $m(m-1) + 1$. This hypothesis is stored in M2_STM.
- It executes Step 3. The Python tool runs the code with $m = 100$ and calculates ($result = 100 * 99 + 1 = 9901$). This is the final **Actual Result** (R).

6.2.3 Step 3: The System 1 / System 2 Interchange (Handling Failure)

Let’s consider an alternative path where the LLM makes a mistake in Step 2 and generates an incorrect closed form, e.g., $m^2 - m + 1$. The plan proceeds, but to be robust, a validation step should exist. The agent could add a step to its plan: “Verify the formula for $m=5$.” The symbolic engine would compute $5^2 - 5 + 1 = 21$. This matches the calculated sequence, so the plan appears valid. It proceeds to calculate $x_{100} = 100^2 - 100 + 1 = 9901$. In this case, the mistake did not lead to an incorrect final answer.

However, imagine the LLM’s first generated code in Step 1 was buggy: ($x[m+1] = x[m-1] + 2*m$).

- The f_{sym} engine would execute this and immediately receive a *KeyError* : 0 from Python when $m = 1$.
- This error is the **Actual Result** (R) of that step. The learning catalyst (η) finds a major discrepancy: an *integer* was expected, but an *Error* was received.
- The control logic (M0) detects this failure. It passes the context—the original code and the error message—back to the System 1 LLM.
- The LLM, now prompted with "The following code failed with a *KeyError*, please fix it," analyzes its mistake and generates the corrected code $x[m]$. This demonstrates the crucial back-and-forth interchange that makes the system robust.

6.2.4 Step 4: System 2 Reflection (The g_{cmp} and g_{upd} Morphisms)

After a successful execution yielding $R = 9901$, the reflection morphisms are triggered.

- The comparison morphism, g_{cmp} , formally evaluates the final state. The form of R (an integer) matches $\hat{E}$. The agent might use an Oracle to validate the correctness of the number 9901. Once validated, the outcome is deemed a success.
- The update morphism, g_{upd} , now constructs the complete KSTAR tuple for this successful experience. It captures the situation, task, the successful multi-step action plan (including the derived formula $m(m - 1) + 1$), the expected result, and the validated actual result.
- This structured KSTAR episode is then stored permanently in the symbolic long-term memory (M2_LTM).

6.3 Two-Level Learning: The Role of the Oracle

What if the agent could not figure out the closed-form solution in Step 2, even after several attempts? This triggers Level 2 learning.

1. The agent's control logic would recognize the repeated failure and invoke its 'Oracle' tool.
2. The Oracle might provide a more abstract hint: "Consider the sum of an arithmetic series."
3. The agent's LLM (System 1) receives this hint. It now re-evaluates the problem from this new perspective. It knows $x_m = x_1 + \sum_{i=1}^{m-1} 2i = 1 + 2\frac{(m-1)m}{2} = 1 + m(m - 1)$.
4. This new, more fundamental derivation is a much more valuable piece of knowledge. The agent validates this new plan (by checking it for a few terms and re-calculating x_{100}).
5. When storing the KSTAR episode in M2_LTM, it now includes the Oracle's hint and the more general, derived strategy. This generalized knowledge is far more powerful for future problems than the simple pattern-matching it attempted initially. This process shows how the agent not only solves the problem but learns a better way to think about the problem class.

7 Theoretical Foundations of NEOLAF and KSTAR

7.1 Functorial Mappings Between Substrate and Representation

Having defined the fundamental categories relevant to the agent ($\mathbb{S}$ for substrate, $\mathbb{R}$ for representations, with subcategories $\mathcal{K}$ and $\mathcal{E}$), we now formalize the bridges between these categories. These bridges are described by functors—structure-preserving mappings between categories that translate objects and morphisms from one domain to another. Specifically, we identify two complementary functors that link the physical and informational domains:

- $\pi : \mathbb{S} \rightarrow \mathbb{R}$, which we will call the **perception functor**.
- $\rho : \mathbb{R} \rightarrow \mathbb{S}$, which we will call the **action functor** (or realization functor).

Intuitively, π handles the mapping from substrate to representation (taking in real-world situations and producing informational descriptions), and ρ handles the mapping from representation to substrate (taking informational content like plans or commands and effecting them in the physical world).

Perception Functor (π). The functor $\pi : \mathbb{S} \rightarrow \mathbb{R}$ formalizes the agent’s perceptual apparatus. For each object X in the substrate category (for example, the state of the world or an environmental entity), $\pi(X)$ is a representation of that state within the agent’s mind (for example, a percept encoded in the agent’s internal format, such as an image encoded as a tensor or a symbolic description of what was observed). For each morphism $f : X \rightarrow Y$ in $\mathbb{S}$ (which might represent a physical process or event that takes the world from state X to state Y), the functor π maps it to a morphism $\pi(f) : \pi(X) \rightarrow \pi(Y)$ in $\mathbb{R}$, which represents the corresponding transformation in the agent’s representations. In other words, if a certain change happens in the environment, π ensures that the agent’s internal representation changes in a corresponding way. For example, if f is the passage of time causing the environment to go from situation X to Y , then $\pi(f)$ might be the update of the agent’s internal belief state from $\pi(X)$ to $\pi(Y)$ reflecting what the agent now perceives. As a concrete case, consider X as an object representing “room is dark” and Y as “room is lit” in $\mathbb{S}$, with f being the action of turning on a light. Then $\pi(X)$ might be a representation like a low-light sensor reading, and $\pi(Y)$ a representation like a bright sensor reading, with $\pi(f)$ being the update in the agent’s perceptual inputs caused by the light turning on. The perception functor thus captures the agent’s sensorimotor interpretation of the world: it defines how raw physical reality is systematically translated into the agent’s internal language of thought.

It is worth noting that in practice π is not a single monolithic function but an aggregate of many sensory processing pathways (visual perception, auditory perception, etc.), each of which transforms some aspect of $\mathbb{S}$ into $\mathbb{R}$. However, by describing it as a functor, we emphasize the principle that this mapping preserves structure: patterns and relationships that exist in the environment (such as spatial or temporal regularities) are reflected as corresponding patterns and relationships in the agent’s representations (such as a spatial map or a timeline of observations). The functorial perspective also implies compositionality: perceiving a sequence of events and then interpreting them is equivalent (up to π) to interpreting each event in turn and composing the internal results.

Action Functor (ρ). The functor $\rho : \mathbb{R} \rightarrow \mathbb{S}$ goes in the opposite direction, from representations to the substrate. It formalizes the agent’s ability to take internal ideas or decisions and make them happen in the physical world through action. For each object M in $\mathbb{R}$ that represents some intended behavior, plan, or constructed artifact, $\rho(M)$ is the corresponding physical manifestation or outcome in $\mathbb{S}$. For instance, if M is an object in $\mathbb{R}$ representing a plan of action (such as a sequence of high-level instructions the agent has formulated), then $\rho(M)$ would be the actual executed action or sequence of movements in the real world that carry out that plan. As another example, if M is a piece of information like “the agent speaks the sentence hello,” then $\rho(M)$ is the physical sound wave produced. For each morphism $g : M \rightarrow N$ in $\mathbb{R}$ (say, a computational process that transforms one internal plan into another, or a decision-making step that revises an intention), $\rho(g)$ is a morphism in $\mathbb{S}$ that represents the corresponding change in the physical domain (for example, the change in executed action sequence or a change in some physical operation).

The action functor underscores the idea that the agent’s actions are not random or ad hoc physical events, but rather are grounded in and driven by internal representations. The structure of an internal plan (such as a plan broken into sub-steps) is mirrored in the structure of the executed behavior (sub-actions achieving those sub-steps). If the agent composes two internal plans into a larger plan (a morphism $g : M \rightarrow N$ in $\mathbb{R}$ could represent “append step B after step A”), then the execution $\rho(g)$ composes the corresponding physical actions in sequence. In this way, ρ preserves the compositionality of plans: executing plan A then plan B yields an overall effect equivalent to executing the composed plan $A-B$. Failures of ρ to be well-defined (for example, if an internal plan cannot be executed due to environmental constraints) manifest as situations where the agent’s representation was incomplete or incorrect—highlighting the need for learning, which we address shortly.

Together, π and ρ allow us to view the agent as an *interpreter* between two worlds. The agent perceives the substrate world via π , constructing experiences in $\mathcal{E}$, and it acts upon the substrate world via ρ , guided by its knowledge in $\mathcal{K}$. If we consider the composite $\pi \circ \rho : \mathbb{R} \rightarrow \mathbb{R}$, this represents a round-trip in which the agent takes some internal representation, realizes it in the world, and then perceives the result back into a (new) representation. Ideally, for a perfect agent with perfect knowledge, $\pi \circ \rho$ on certain parts of $\mathbb{R}$ (like a well-planned action) might be close to the identity mapping—meaning the agent sees the world conform to its expectations. In general, however, $\pi \circ \rho$ will not be identity: the difference $\pi \circ \rho(M)$ vs. M represents the agent’s surprise or prediction error, and drives learning as we will formalize below.

7.2 Functors for Planning and Learning: $\mathbf{P}$ and $\mathbf{L}$

While π and ρ handle interfacing between the agent’s mind and the world, the core cognitive algorithms of the agent are captured by processes entirely within the representational domain. Chief among these are the processes of *planning* (or decision-making) and *learning* (updating knowledge from experience). We formalize each of these as a functor between the subcategories defined earlier:

- $\mathbf{P} : \mathcal{K} \rightarrow \mathcal{E}$, the **planning functor**, which generates an anticipated experience (including an action plan and its expected outcome) from the agent’s current knowledge state.
- $\mathbf{L} : \mathcal{E} \rightarrow \mathcal{K}$, the **learning functor**, which assimilates an experience (in particular, the actual observed outcome of an action) into the agent’s knowledge, producing an updated knowledge state.

These functors encapsulate the knowledge–experience duality at the heart of the KSTAR framework: $\mathbf{P}$ projects knowledge into a hypothesized experience (what the agent *thinks* will happen or should happen if it acts in a certain way), and $\mathbf{L}$ projects an experience into a refinement of knowledge (what the agent *learns* from what actually happened). By composing $\mathbf{P}$ and $\mathbf{L}$, we will obtain an endofunctor on $\mathcal{K}$ that represents a full cycle of thinking, acting, and learning.

Planning Functor ($\mathbf{P}$). $\mathbf{P} : \mathcal{K} \rightarrow \mathcal{E}$ takes the agent’s knowledge and produces a planned experience. More concretely, given an object $K \in \mathcal{K}$ (representing the agent’s current knowledge state, along with perhaps its current goal or task context embedded in that knowledge), $\mathbf{P}(K)$ is an object in $\mathcal{E}$ that represents the agent’s *predicted or intended experience* when it acts based on that knowledge. Typically, $\mathbf{P}(K)$ will consist of two parts: an *action plan* (a sequence of actions the agent decides to perform) and an *expected result* that the agent anticipates will occur if those actions are executed in the current situation. For example, if the agent’s knowledge K includes understanding of how to cook an egg and the task is “cook an egg,” then $\mathbf{P}(K)$ might be an experience object describing the imagined sequence of steps (crack egg, turn on stove, etc.) and the expected outcome (a cooked egg on a plate). This $\mathbf{P}(K)$ is not yet a real experience, but a *simulated or projected experience*—in effect, a scenario that the agent pulls from its knowledge, customizing it to the current situation.

For a morphism $h : K_1 \rightarrow K_2$ in $\mathcal{K}$ (say h is a knowledge update or a logical inference that changes the agent’s beliefs from state K_1 to K_2), the planning functor produces a morphism $\mathbf{P}(h) : \mathbf{P}(K_1) \rightarrow \mathbf{P}(K_2)$ in $\mathcal{E}$. This $\mathbf{P}(h)$ represents the corresponding transformation in the anticipated experiences. For instance, if h added a new piece of knowledge that “the stove is broken” to go from K_1 to K_2 , then $\mathbf{P}(h)$ would transform the old plan (perhaps a plan that assumed a working stove) into a new plan in $\mathcal{E}$ (maybe a revised plan anticipating a different method to cook the egg, or expecting failure). In general, $\mathbf{P}$ being a functor means that it preserves composition of knowledge updates: if one were to apply two knowledge transformations in sequence and then plan, the result is the same as planning after each knowledge change step by step. This ensures consistency in how changes in knowledge affect changes in anticipated experiences.

One can think of $\mathbf{P}$ as the agent’s “imagination” or predictive model: it uses what the agent knows to forecast what might happen in a given context. It is through $\mathbf{P}$ that the agent generates expectations (in KSTAR terms, the Expected Result E in a KSTAR tuple) and intentions (the planned Action A). However, those expectations may or may not match reality, which is where the next functor comes into play.

Learning Functor ($\mathbf{L}$). $\mathbf{L} : \mathcal{E} \rightarrow \mathcal{K}$ takes an experience and produces a change in knowledge. Given an object $E \in \mathcal{E}$ (representing an actual experience the agent has gone through—i.e. a situation, action, and the actual observed result R that occurred), $\mathbf{L}(E)$ is an object in $\mathcal{K}$ representing the *knowledge update* or learning derived from that experience. In practice, $\mathbf{L}(E)$ might be a structured representation of what was learned: for example, a generalization or an adjustment of a model parameter, or a newly stored rule like “if stove is broken, microwaving can cook an egg” derived from the experience. Importantly, $\mathbf{L}(E)$ is not merely the raw experience itself stored in memory, but rather the distilled knowledge extracted from it—often taking the form of a delta or refinement to the prior knowledge state.

For a morphism $t : E_1 \rightarrow E_2$ in $\mathcal{E}$ (perhaps t represents one experience happening after another, or one experience’s context leading to another’s), the learning functor yields $\mathbf{L}(t) : \mathbf{L}(E_1) \rightarrow \mathbf{L}(E_2)$ in $\mathcal{K}$, describing how the knowledge updates relate. If t is a morphism indicating, say, that E_2 is a subsequent experience building on E_1 (perhaps the agent tried something, got a result E_1 , then

adjusted and tried again leading to E_2), then $\mathbf{L}(t)$ would map the learning from the first experience to the learning from the second. Functoriality of $\mathbf{L}$ implies that if multiple experiences accumulate, the total knowledge gained is the same whether the agent learns incrementally after each experience or in batch after all—an important property of consistency. It also suggests systematicity: similar experiences yield similar knowledge updates, reflecting an underlying regularity in how the agent learns.

In summary, $\mathbf{P}$ and $\mathbf{L}$ formalize the two halves of the thought-action cycle: projecting knowledge forward to predict outcomes (planning), and integrating outcomes backward to improve knowledge (learning). They are dual to each other: $\mathbf{P}$ goes from $\mathcal{K}$ to $\mathcal{E}$ (knowledge to expected experience), and $\mathbf{L}$ goes from $\mathcal{E}$ to $\mathcal{K}$ (experience to updated knowledge). In the next section, we will see how their composition closes the loop, and how the discrepancy between the $\mathbf{P}$ prediction and reality becomes the driver of learning, captured via a natural transformation.

7.3 Natural Transformations and the Experience–Learning Loop

When the agent acts and then learns, we have two parallel mappings from the context of action to the outcome: one mapping is the agent’s own prediction (via $\mathbf{P}$ composed with action execution), and the other is the actual observation (via the environment and then $\mathbf{L}$). Category theory provides a convenient notion to compare two such mappings: a natural transformation between functors. In NEOLAF, we identify a critical natural transformation, denoted α or here η (we will use α for clarity in this context), which encapsulates the alignment (or misalignment) between what the agent expected and what actually occurred.

Consider the agent’s state just before executing a plan. We can define a category of “contexts” $\mathcal{X}$, whose objects $x \in \mathcal{X}$ represent all relevant information about the situation before an action (for instance, x could be a tuple (K, S, T) consisting of the agent’s knowledge state, the current situation, and the task or goal at hand). We also define a category of “outcomes” $\mathcal{O}$, whose objects are possible results or outcome states after an action. Now we can imagine two functors from $\mathcal{X}$ to $\mathcal{O}$:

- A predictive functor $F : \mathcal{X} \rightarrow \mathcal{O}$, which given a context x produces the agent’s expected outcome $F(x)$ (this functor is effectively the composition of the planning functor with the action functor and the perception of the anticipated result; it encodes what the agent *thinks* will happen in $\mathcal{O}$ based on x).
- An observational functor $G : \mathcal{X} \rightarrow \mathcal{O}$, which given the same context x produces the actual outcome $G(x)$ after the agent’s action is executed and observed (this functor encapsulates what *actually* happens: it involves applying ρ to execute the plan in the real world and then π to perceive the actual result back into a representation, and finally mapping that real experience into an outcome representation).

For each context $x \in \mathcal{X}$ (for example, $x = (K_t, S_t, T_t)$ capturing the agent’s knowledge, situation, and task at time t), $F(x)$ is the expected result representation $S_{t+1}^{(e)}$ (what the agent expects the next state S_{t+1} will be, according to its plan), and $G(x)$ is the actual result representation $S_{t+1}^{(a)}$ (what actually happened in the next state, as represented internally after observation). In an ideal world, $F(x)$ and $G(x)$ would coincide for all x —the agent’s predictions would always match reality. In practice, there will often be a discrepancy.

We capture that discrepancy with a natural transformation $\alpha : F \Rightarrow G$. A *natural transformation* in category theory is a family of morphisms that connects two functors, one for each object in the domain category, such that a certain coherence condition holds. Here, α assigns to each context $x \in \mathcal{X}$ a morphism $\alpha_x : F(x) \rightarrow G(x)$ in the outcome category $\mathcal{O}$. This morphism α_x can be thought of as the “correction” or “error signal” transforming the predicted outcome into the actual outcome. The naturality condition requires that for any morphism $u : x \rightarrow y$ in $\mathcal{X}$ (i.e., an evolution or change from context x to context y , for example time progression or a shift in internal state before action), the following holds: $G(u) \circ \alpha_x = \alpha_y \circ F(u)$. In intuitive terms, this means that whether we update the context first and then compute the prediction-vs-actual difference, or compute the difference first in context x and then map it forward, we get a consistent result. This is a formal way of saying the prediction errors line up properly with respect to contextual changes, ensuring the agent’s learning process is well-behaved when contexts shift.

In NEOLAF’s terminology, this α is referred to as the “alpha factor” or α -natural transformation factor. It is a crucial quantity: if α_x is the identity (or trivial) morphism, the predicted and actual outcomes coincided perfectly for context x , meaning the agent’s knowledge was validated in that instance. If α_x is a significant deviation, it indicates a surprise or prediction error that signals a learning opportunity. The larger or more complex the α transformation needed to convert $F(x)$ to $G(x)$, the more the agent needs to adjust its knowledge to account for the new information.

Now we connect this back to our earlier functors $\mathbf{P}$ and $\mathbf{L}$. After the agent observes the actual outcome $G(x)$, it will invoke the learning functor $\mathbf{L}$ to incorporate this outcome into its knowledge. The learning functor $\mathbf{L}$ maps the experienced outcome (an object in $\mathcal{E}$ corresponding to $G(x)$) to a knowledge update ΔK in $\mathcal{K}$. If K_t was the prior knowledge, the new knowledge K_{t+1} can be seen as K_t updated by ΔK : symbolically, $K_{t+1} = K_t \oplus \mathbf{L}(G(x))$ (where $\oplus$ indicates integration of the learned information). On the other hand, we can consider how the expected outcome would have informed knowledge if it had been correct: $\mathbf{L}(F(x))$ would be the hypothetical knowledge change if the world behaved exactly as predicted. The natural transformation $\alpha : F \Rightarrow G$ thus links these two: it provides the “difference” that can be applied to $\mathbf{L}(F(x))$ to yield $\mathbf{L}(G(x))$. In essence, α_x tells the agent how to adjust the learning derived from its expectation to match the learning needed for reality.

While α captures the moment-to-moment prediction error, there is another important natural transformation in our formalism, which we will denote η . This η emerges in the context of treating the overall experience-to-knowledge loop as a kind of *monad* or self-contained learning operation on the space of knowledge states. Consider the composition $\mathbf{T} = \mathbf{L} \circ \mathbf{P}$, which is a functor $\mathbf{T} : \mathcal{K} \rightarrow \mathcal{K}$ (taking a knowledge state, projecting to an expected experience, then learning from that “experience”). $\mathbf{T}(K)$ is essentially the agent’s would-be new knowledge if it trusted its prediction as actual experience. In reality, the actual operation is $K \rightarrow K'$ via the real experience, which we might denote as $\mathbf{U}(K)$ (the actual update after executing in the environment). If the agent never acted (no new experience), the knowledge would remain K ; we have the identity functor $\text{Id}_{\mathcal{K}}$ representing “do nothing, change nothing.”

The natural transformation $\eta : \text{Id}_{\mathcal{K}} \Rightarrow \mathbf{L} \circ \mathbf{P}$ can be interpreted as the **unit of the learning monad**: it injects a given knowledge state K into the learning process in a way that is consistent with the functor composition. $\eta_K : K \rightarrow (\mathbf{L} \circ \mathbf{P})(K)$ is a morphism in $\mathcal{K}$ that represents the trivial learning achieved by “planning an experience without actually gaining new information.” In practical terms, η_K might correspond to recording an entry of knowledge that simply reiterates what was already known (since if we plan and then immediately pretend we learned from that plan, we gain nothing new aside from perhaps confirming our expectations). The purpose of defining η formally is to satisfy the monadic axioms if we treat $(\mathbf{T} = \mathbf{L} \circ \mathbf{P}, \eta, \mu)$ as a monad on $\mathcal{K}$. The other natural transformation $\mu : \mathbf{T} \circ \mathbf{T} \Rightarrow \mathbf{T}$ (the monadic multiplication) would then handle the composition of two learning cycles in sequence, flattening them into one (ensuring that if you treat the result of one full cycle as input to another, it’s the same as a two-step learning in one go). The details of μ aside, the inclusion of η underscores that a knowledge state can always be viewed as a special case of an experience-updated knowledge state (one with no change), providing an identity element for learning.

By leveraging these category-theoretic constructs—functors for perception, action, planning, and learning, and natural transformations for aligning expectations with outcomes—we obtain a high-level but rigorous description of the agent’s cognitive dynamics. We can now summarize the entire cycle of operation as a composition of morphisms and functors, which we call the KSTAR experience-to-learning loop:

$$K_t \xrightarrow{f_{\text{plan}}} P_t \xrightarrow{f_{\text{exec}}} A^{\text{exec}}t \xrightarrow{f_{\text{obs}}} E_t \xrightarrow{f_{\text{learn}}} K_{t+1}. \quad (1)$$

In this chain, $f_{\text{plan}} : (K_t, S_t, T_t) \rightarrow P_t$ is the act of formulating an action plan (an element of $\mathcal{E}$, since it’s a planned experience) from the current knowledge K_t and context (situation S_t and task T_t) – this corresponds to $\mathbf{P}(K_t)$ conceptually. Next, $f_{\text{exec}} : P_t \rightarrow A^{\text{exec}}t$ is the execution of that plan in the substrate environment (applying ρ to get a physical action or effect $A^{\text{exec}}t \in \mathbb{S}$). Then $f_{\text{obs}} : A^{\text{exec}}t \rightarrow E_t$ is the perception and interpretation of the actual outcome (applying π to produce an experienced result $E_t \in \mathcal{E}$, which may differ from the planned expectation). Finally, $f_{\text{learn}} : (E_t, K_t) \rightarrow K_{t+1}$ is the incorporation of that experience into knowledge (corresponding to $\mathbf{L}(E_t)$ applied to K_t , yielding updated knowledge K_{t+1}). The composite $f_{\text{learn}} \circ f_{\text{obs}} \circ f_{\text{exec}} \circ f_{\text{plan}} : K_t \rightarrow K_{t+1}$ is the overall effect of one cycle of the KSTAR loop on the agent’s knowledge. Ideally (in the limit of many cycles and sufficient exploration), this becomes an *identity morphism* or at

least stabilizes for a rational agent – meaning the knowledge eventually converges such that further experience doesn’t drastically change it except in novel situations. In category theoretic terms, one could say the agent aims for $\mathbf{L} \circ \mathbf{P}$ to act as the identity on $\mathcal{K}$ for experiences within its learned domain, and $\alpha : F \Rightarrow G$ tends toward an identity transformation (accurate predictions) in well-understood contexts.

Placeholder for a diagram depicting the KSTAR cycle as a commutative diagram or loop: showing K_t going through functor $\mathbf{P}$ to an expected experience, then via action in $\mathbb{S}$ and perception back to an actual experience, and then via $\mathbf{L}$ to K_{t+1} . The natural transformation α might be indicated between the expected and actual result, highlighting the discrepancy used for learning.

Figure 4: Category-theoretic depiction of the KSTAR experiential learning loop. The agent’s knowledge state K_t generates an action plan and expected outcome ($\mathbf{P}$ functor). The plan is executed in the substrate world (ρ action functor), producing an actual result which is perceived back into an experience (π perception functor). The experience is then integrated into knowledge ($\mathbf{L}$ functor), yielding the updated knowledge K_{t+1} . A natural transformation α bridges the expected outcome and actual outcome, representing the prediction error that drives learning.

Thus far, we have established a theoretical foundation for NEOLAF and the KSTAR framework in abstract terms. We have an ontology that splits reality into substrate and representation, a formal definition of knowledge and experience, and functorial mappings that describe perception, action, planning, and learning. These abstractions are powerful because they ensure that our architecture has a principled organization: each component of the agent’s loop has a well-defined input, output, and relation to other components. Moreover, properties like functoriality and naturality guarantee consistency — for example, that the agent’s learning behavior is systematic and structure-preserving, rather than ad-hoc. In the next section, we will transition from this high-level formal perspective to the concrete implementation of these ideas in the NEOLAF architecture, showing how neural-network-based learning (transformers) and symbolic reasoning components are combined to instantiate the functors $\mathbf{P}$ and $\mathbf{L}$ and to realize the overall cognitive cycle described above.

7.4 Neuro-Symbolic Integration in the KSTAR Architecture

While category theory provides a unifying abstract view of the learning cycle, a practical cognitive architecture must specify how these functors and transformations are actually realized by algorithms and data structures. NEOLAF (Never-Ending Open Learning Agent Framework) is a *neuro-symbolic* architecture, meaning it integrates neural network-based learning (which excels at pattern recognition and function approximation) with symbolic reasoning (which excels at structured manipulation of explicit knowledge). This integration is done in service of implementing the KSTAR cycle in a robust, scalable way. In this section, we explain how the neural and symbolic components work together, and how the formal elements defined above correspond to specific processes in NEOLAF. We will highlight a two-stage cognitive execution process (plan-generation and execution/validation), discuss the advantages of marrying neural and symbolic methods, and describe mechanisms like multi-agent delegation and cognitive pattern stability which emerge naturally from the NEOLAF design.

7.4.1 Two-Stage Cognitive Processing: Plan Generation and Enactment

A hallmark of the NEOLAF/KSTAR approach is that each cycle of interaction with the environment is broken into (at least) two distinct stages, which we can call **plan generation** and **plan enactment (with validation)**. These correspond roughly to first running the functor $\mathbf{P}$ (entirely inside the agent’s mind, using its knowledge to produce an internal plan and prediction) and then running the sequence ρ (action) + environment + π (observation) + $\mathbf{L}$ (learning) to actually carry out the plan and update knowledge from the real outcome.

In NEOLAF’s implementation, the **plan generation stage** is primarily handled by a Transformer-based neural network (e.g. a Large Language Model, LLM) that plays the role of an advanced *reasoning engine or policy generator*. The LLM is given as input a description of the current context: it is essentially “fed” the relevant elements of (K_t, S_t, T_t) in some interpretable form (for example, a prompt that includes the agent’s current knowledge or beliefs, a description of the situation from the agent’s perceptions, and the current task or goal). Utilizing its massive learned parameters (which

encode a wealth of patterns from training data), the LLM then produces an output that represents the *planned experience* $\mathbf{P}(K_t)$. Concretely, this output might be formatted as a structured plan, such as a step-by-step list of actions to take (the “Action” part of KSTAR), along with a statement of the expected outcome (the “Expected Result” part of KSTAR). The LLM is effectively functioning as a highly flexible, learned policy planner or “semantic compiler.” It compiles the high-level goal and context into a concrete, low-level action script. This script can be in a human-readable form (like pseudo-code or natural language instructions) or in a machine-executable form (like a domain-specific programming language, JSON or YAML plan, or API calls to tools). What is crucial is that the LLM uses its neural knowledge (associative, statistical knowledge gleaned from vast data) to produce a plan that not only is plausible in the given context but also is expressed in a structured format that the next stage can work with.

This first stage leverages the neural component’s strengths: handling ambiguity, drawing on broad prior knowledge, performing analogical reasoning (“this situation is like that previous situation, so a similar approach might work”), and generally dealing with the inherent uncertainty and creativity required in open-world problem-solving. The LLM is adept at “filling in gaps” and proposing actions even in novel situations where explicit rules might not exist, because it can generalize from patterns it has seen. It is, however, a learned model and not guaranteed to be correct or optimal—it might produce actions that are suboptimal or even incorrect, or predict outcomes that turn out to be wrong. This is expected and acceptable, because the second stage of processing will catch and correct any discrepancies.

The **plan enactment and validation stage** is where the symbolic and environment-interfacing components come into play, corresponding to the f_{exec} , f_{obs} , f_{learn} steps (or ρ , environment dynamics, π , and $\mathbf{L}$ in functor terms). Once the LLM has generated a candidate plan P_t (with expected result), this plan is handed over to either a symbolic executor or directly to the environment to be carried out. “Symbolic executor” here can mean a few things depending on the domain: it could be a logic-based module that interprets the plan steps (for instance, a theorem prover if the plan involves proving something, or a task scheduler if it involves high-level tasks), or it could be a straightforward interpreter (for example, if the LLM outputs a piece of Python code as a plan, a Python interpreter acts as the executor). In many cases, the environment itself will be the executor — for example, if the agent is a robot and the plan includes a sequence of movements, the agent’s motor control system will execute those movements in the physical world.

During execution, the agent (via its sensors, represented by π) observes what actually happens. If the plan was output as code or a formal script, it may contain calls to get feedback (like sensor readings or sub-computations). The symbolic component might also explicitly monitor the execution, checking conditions or invariants (for instance, a symbolic rule might assert “if step 3 of the plan fails, then abort and flag an error”). This monitoring is part of what we call validation: ensuring that the outcomes of each step match expectations and that any deviations are noted.

At the end of execution, the agent has an actual outcome R_{actual} (the Actual Result in KSTAR terms). Now the comparison between expected and actual happens. If the actual result matches the expected result that the LLM predicted, then the plan is validated and was presumably correct (in which case the agent might simply reinforce its knowledge that “this plan works as expected in this context”). If there is a mismatch, then we have a non-trivial α transformation (prediction error) which must be analyzed. The agent (likely with help of the symbolic side, possibly guided by the neural side) computes the difference: for example, “Expected the egg to be cooked, but it is still raw,” or “Expected the API call to return a value, but it returned an error.” This discrepancy is then used to adjust the knowledge.

The knowledge update stage can be performed by a combination of symbolic logic and neural processing. For instance, if the result was a factual outcome contrary to expectation, a symbolic rule might be used to store a new fact or a counterexample. Alternatively, the LLM itself can be prompted with a description of the expected vs actual and asked to explain the difference or generalize from it, effectively performing analogical or causal reasoning to update its internal model (or an external memory). NEOLAF often uses a structured memory (the KSTAR long-term memory, M2L) to archive the entire episode ($K_t, S_t, T_t, A_t, E_{\text{expected}}, R_{\text{actual}}$, analysis) as a new piece of knowledge. The learning functor $\mathbf{L}$ in practice might be realized by the LLM producing a concise summary or rule from the episode that can be stored symbolically. For example, after a failed plan, the system could prompt the LLM: “What went wrong and what should be learned?” and the LLM might respond

with: “The stove was broken, so the egg couldn’t cook; the agent should learn to check if the stove is operational before cooking.” This summary is then stored as new knowledge (perhaps as a rule: “if stove is broken, cooking will not succeed” or as a more general heuristic).

To put it succinctly, the two-stage process divides cognition into an *imaginative phase* and an *experiential phase*. The imaginative phase (neural-dominated) is fast, parallel, and uses prior data to propose solutions (aligning with what psychologists call System 1 thinking). The experiential phase (symbolic+environment) is grounded, sequential, and meticulous, checking against reality and updating structured records (analogous to System 2 deliberation and verification). This division ensures that the agent neither relies purely on guesswork (because guesses are checked against reality) nor gets stuck in rigid rule-following (because creative new plans can be generated even without a pre-written rule). It marries the best of both worlds: the efficiency and generality of neural inference with the precision and clarity of symbolic execution.

7.4.2 Synergy of Neural and Symbolic Components

The neuro-symbolic approach in NEOLAF yields a number of significant advantages for the learning agent, directly addressing limitations that either approach (neural-only or symbolic-only) would face on its own. Here we highlight several key benefits of this integration:

Robust Multi-Step Reasoning. Large neural models (LLMs) are universal function approximators that have absorbed vast amounts of patterns, including logical and stepwise patterns, from their training. By guiding an LLM to output not just an answer, but a *chain of steps* (the action plan), we force it to externalize its reasoning. This has two effects: (1) it allows the symbolic system to verify each step of the reasoning chain, catching errors or implausible leaps (mitigating the classic problem of neural “hallucinations” or reasoning mistakes that are hidden in the latent computation), and (2) it provides transparency and interpretability, since the plan is in a human or at least machine-readable form. The symbolic side, with knowledge of logic and constraints, can enforce that the plan follows certain rules (for example, an internal rule might forbid the agent from performing an action that would cause self-damage; if the LLM-proposed plan violates that, a symbolic check can flag it before execution). Thus, multi-step reasoning becomes more reliable: the neural component proposes, the symbolic component disposes (or approves).

Continuous Learning from Novel Situations. A purely symbolic system struggles with novel inputs that weren’t anticipated by its rules, while a purely neural system typically has a fixed model after training and can’t easily incorporate new facts (short of retraining). In NEOLAF, when a novel situation occurs (say the agent encounters a scenario its creators never programmed or an out-of-distribution event relative to its training), the neural component can still attempt something by analogy (because neural nets excel at interpolating or making do with what they know), and whether it succeeds or fails, the result is turned into a symbolic knowledge increment. Symbolically, the agent’s knowledge base expands with each new lesson (the KSTAR episodes in memory), meaning the agent becomes explicitly aware of the new situation for next time. Meanwhile, the neural net can also be updated or fine-tuned gradually using these experiences (for instance, important new data can be used to fine-tune the LLM’s weights or to condition its future prompts). The combination ensures that learning is both *deep* (affecting the neural representation space) and *surface-level* (affecting explicit recallable facts), leading to a more resilient learner that improves with each interaction.

Reduction of Hallucination and Grounding of Knowledge. LLMs sometimes “hallucinate” incorrect facts or overly confident answers because they lack a grounding in actual environment interaction. By coupling the LLM with real feedback from the world (through execution and observation), we create a closed feedback loop that corrects false beliefs. The α natural transformation precisely measures the hallucination gap: if the LLM expected something that didn’t happen, that discrepancy is fed back as an error signal. Over time, with sufficient feedback, the neural model can adjust to avoid repeating those errors, and symbolically the system might mark certain knowledge as “unverified” or “disproven.” Additionally, the agent can be designed to become skeptical of its own ungrounded assertions: if a particular type of query often leads to large α adjustments, the system may learn to consult an external tool or ask for clarification rather than hallucinate. The presence of a symbolic memory of past cases helps here: the agent can recall, “Last time I assumed X and it was wrong,” which tempers its future responses.

Explainability and Inspectability. One major advantage of symbolic systems is that their reasoning process can be inspected and explained. In NEOLAF, even though an LLM might generate the plan,

the plan is in a form that a human or another machine can examine. Furthermore, the KSTAR record that gets stored (with situation, task, action, expected vs. actual result, and analysis) serves as an explanation of what happened and why the agent learned what it learned. If needed, the agent can be queried to produce a justification for its actions by referencing these stored structures (“I did action A because I expected X to happen, based on knowledge K; when instead Y happened, I updated my knowledge accordingly.”). This goes a long way toward making AI behavior interpretable and trustworthy, a critical requirement in many domains (such as healthcare or finance) where pure neural networks are often seen as “black boxes.”

Compositional Generalization. Symbolic representations naturally afford compositionality—complex structures built from simpler parts—which is something neural networks can struggle with (though they are good at pattern matching). By ensuring that the agent’s internal communication and memory use a structured format (for example, representing plans as structured graphs or code), NEOLAF enables the agent to recombine known pieces in new ways. An LLM might never have seen a very specific situation before, but if it has a symbolic language of thought (like the KSTAR representation) it can piece together known components (a known method applied to a new object, etc.). The symbolic memory acts like a library of building blocks that the neural planner can draw upon. In effect, the LLM is “prompted” not just with raw text, but with a curated, formal set of past experiences relevant to the task at hand, which can dramatically improve systematic generalization.

Multi-modal and Multi-domain flexibility. The architecture does not hard-code any particular modality or domain: the functors and categories are abstract, meaning the same design can encompass vision, language, robotics, etc. For example, π might include a computer vision module for images and a parsing module for text, funneling both into a unified representation language. The LLM can handle language-based reasoning, while mathematical or highly precise operations could be delegated to external symbolic solvers (the LLM just needs to know how to call them). This flexibility to delegate brings us to the next topic: multi-agent and multi-module delegation.

7.4.3 Multi-Agent Delegation and Communication

The NEOLAF architecture, especially in its planning stage, is inherently capable of orchestrating multiple agents or modules in a coherent way. This stems from the fact that the output of the neural planner can be not just a single-agent plan, but a network of sub-tasks assigned to various specialized units. We sometimes describe this as an **agent society** or a multi-agent layer within the architecture.

For example, imagine the agent has access to a suite of specialized tools or sub-agents: one might be a database query engine (SQL agent), another might be a web-scraping agent, another a robotic arm controller, etc. The LLM, when formulating a plan, can incorporate these as actors. A plan might look like: “First, use the `csv_cleaner` agent to preprocess the data; then have the `analyzer` agent run statistical analysis; finally, have the `reporter` agent summarize the results.” Each of these steps can be formatted in a domain-specific language (DSL) or protocol message. For instance, the plan might literally include a snippet of code or a structured message:

```
task:
type: transform_data
target_agent: csv_cleaner
parameters:
file: "sensor_logs.csv"
method: "interpolate_missing"
```

This kind of structured directive, which could be in YAML or JSON, is interpretable by the agent’s subsystem responsible for delegation. It essentially tells a sub-agent (here `csv_cleaner`) what to do with what parameters. The use of a symbolic, structured format for inter-agent communication is a form of qualitative representation that persists in the architecture even though the reasoning to create it was done by a neural model. It ensures that multiple agents can coordinate: the output of one is well-defined and becomes the input of another. The *multi-agent communication layer* in NEOLAF is therefore a natural extension of the planning functor $\mathbf{P}$: instead of $\mathbf{P}$ yielding a single action plan, it yields a coordinated set of actions by various actors, all encoded symbolically.

The advantage of this approach is significant. The agent can leverage the expertise of specialized modules without those modules needing to be intelligent in the human sense—they just need to do their narrow job well (like a database engine executes queries efficiently). The LLM’s broad

intelligence is used to orchestrate these narrow intelligences. We effectively get a system that is more than the sum of its parts: the neural planner is the glue that can dynamically form teams of tools to solve a problem, something a static symbolic system would find hard (it would require a lot of pre-defined rules on how to use each tool). The neural model, having been trained on code, documentation, and human examples, often “knows” how to call upon these tools—because it has seen analogous patterns in its training data (for instance, how humans delegate tasks to functions or APIs).

Delegation through symbolic communication also contributes to **cognitive pattern stability**. Each sub-task is expressed in a standard format; these formats change little even as tasks vary. For example, regardless of the content of the CSV file or the specifics of missing data interpolation, the plan snippet for using the `csv_cleaner` agent might remain structurally the same. This means the agent’s cognition has stable, reusable patterns (like templates) for common tasks, which it can fill in appropriately. Stability comes from consistency: the agent reuses known successful sub-plans in similar contexts.

Furthermore, the natural transformation formalism can be extended to multi-agent contexts: each sub-agent could have its own predictive vs actual outcome that the orchestrator monitors. In category terms, one might consider functors for each agent and natural transformations aligning expected and actual at the level of each sub-task. The orchestrating agent (the LLM planner) then has to compose these. The mathematics of functor composition and natural transformations gives a principled way to reason about such compositions, ensuring that errors in sub-tasks propagate correctly to the top-level error signal for learning.

7.4.4 Stability of Cognitive Patterns and Memory

Finally, a critical aspect of NEOLAF is its emphasis on building up a stable set of cognitive patterns over time. By “cognitive pattern stability,” we refer to the architecture’s ability to form enduring structures of thought that persist and become refined rather than reinvented from scratch in each instance. This contrasts with a naïve LLM that might approach each query anew, without accumulating a long-term structured memory of how it solved previous problems.

Several features of NEOLAF contribute to this stability:

- **Structured Episodic Memory (KSTAR records):** Each experience is stored in a structured form (Knowledge, Situation, Task, Action, Result). These records themselves form a growing graph or network of interconnected experiences. When a new situation arises, the agent can retrieve a similar past case and adapt its plan accordingly. This is effectively a case-based reasoning approach, enhanced by neural pattern matching to find relevant cases. The structure ensures that what is retrieved is meaningful and can be directly applied or modified. Over time, as more cases accumulate, the agent’s responses become more consistent because it is referencing actual precedents in its own experience rather than relying purely on the unpredictability of generative guessing.
- **Internal Language of Thought:** The use of a consistent internal format for states, tasks, and actions (for example, representing tasks in a quasi-programmatic form) means the agent is effectively developing its own “mental language.” Because the LLM both reads and writes this language (it reads the context in that format and outputs plans in that format), the language remains stable and evolves slowly. It’s analogous to how humans develop jargon or standardized procedures in a domain—once established, those patterns are reused for efficiency and clarity. For the agent, this means less likelihood of off-the-wall outputs; it will tend to conform to its learned internal schemas.
- **Reinforcement of Successful Patterns:** When a plan works well and yields minimal α error (meaning expected matched actual), that instance can strengthen the pattern. The agent can either explicitly mark that plan as verified or the neural model can be subtly adjusted (via a form of reinforcement learning or fine-tuning) to increase the probability of generating similar plans in the future. Conversely, when a plan fails, the changes made in knowledge will discourage repetition of the same flawed approach. This feedback creates attractors in the agent’s cognitive state space—stable patterns that represent strategies or concepts that have been validated repeatedly.
- **Modularization of Knowledge:** NEOLAF’s design, with distinct memory layers (M0, M1, M2L, etc. as mentioned in its documentation), encourages the agent to compartmentalize

knowledge. For example, M2L might store factual knowledge, while M1 (neural memory) stores more subconscious patterns. By keeping certain core knowledge (M0) fixed and providing a backbone, the agent ensures that fundamental patterns (like basic reasoning skills, or innate biases) remain stable while only higher-level knowledge expands. This mitigates catastrophic forgetting; new experiences add onto existing structures instead of overwriting them indiscriminately.

- **Cognitive Economy via Symbols:** Symbolic representations allow the agent to compress and abstract away details. If a recurring pattern is noticed across many experiences, the agent can create a new symbolic concept for it, which then gets used in place of re-deriving the pattern each time. This conceptual chunking leads to more stable higher-level patterns of thought because the agent starts treating complex sequences as single units once they are mastered. In category theory terms, this could be seen as forming quotients or abstractions of the category of experiences: multiple different low-level episodes might map to one high-level “concept morphism” in the knowledge category.

In sum, cognitive pattern stability is about the agent becoming *predictable to itself* (and to observers) in the sense that it develops a repertoire of known strategies and representations. NEOLAF fosters this by ensuring that every loop of experience not only updates knowledge but also refines how that knowledge is organized and expressed. Over time, the agent transitions from learning basic truths to learning about its own learning process—meta-learning—such that it can adjust its approach to problem solving in a stable, convergent way.

Placeholder for a diagram illustrating the neural-symbolic architecture: e.g., a flowchart showing an LLM taking input and producing a plan, symbolic modules executing the plan, and feedback returning to a memory store. Possibly highlight multiple sub-agents and the stable representational format of their communication.

Figure 5: Neuro-symbolic architecture of NEOLAF implementing the KSTAR loop. The Transformer-based neural module generates plans and predictions from the agent’s knowledge and current input. The symbolic execution module (and environment) carries out these plans, and the results are observed and compared to expectations. New experiences are encoded into symbolic memory, and feedback is used to adjust both the explicit knowledge base and the implicit neural patterns. Stable communication formats and modular sub-agent orchestration are depicted, illustrating multi-agent delegation and consistent cognitive patterns.

7.5 Conclusion

In this chapter, we provided a comprehensive theoretical foundation for the NEOLAF architecture and its KSTAR learning framework. By leveraging the abstractions of category theory, we described how an AI agent can be formally understood as a system of functors and transformations bridging the physical and informational realms. The ontology of substrate vs. representation set the stage for distinguishing what it means for an agent to “experience” something (a change in $\mathbb{S}$ perceived as information in $\mathbb{R}$) and to “know” something (a structured representation in $\mathcal{K}$ that persists and informs future actions). We defined the key categories $\mathbb{S}$ and $\mathbb{R}$, with knowledge $\mathcal{K}$ and experience $\mathcal{E}$ as subcategories capturing the agent’s mental state in terms of enduring understanding versus transient observations. Upon this scaffold, we introduced functors π , ρ , $\mathbf{P}$, and $\mathbf{L}$, which model perception, action, planning, and learning, respectively, thereby formalizing how an agent interconverts physical happenings and mental constructs. Natural transformations, especially the α factor, were shown to play a pivotal role in representing the misalignment between expectation and reality, driving the update of knowledge. The entire KSTAR loop was thus recast as a composition of categorical morphisms, illuminating its systematic and principled nature (and hinting at a monadic structure that ensures the learning process is integrative and self-consistent).

We then bridged the gap from this formal understanding to a practical neuro-symbolic architecture. By dividing cognition into a neural-driven imaginative phase and a symbolic-grounded execution phase, NEOLAF capitalizes on the strengths of both paradigms. We saw how a Transformer-based planner can act as a powerful functor instantiation for $\mathbf{P}$, mapping knowledge to planned experience, and how the environment together with symbolic logic completes $\mathbf{L}$ by yielding actual experience and

assimilating it. The advantages of this hybrid approach are manifold: improved multi-step reasoning, continuous learning from feedback, reduced hallucination via grounding, enhanced transparency, and better generalization through compositional and analogical reuse of knowledge. We also discussed the architecture’s inherent ability to manage multiple sub-agents and tools, delegating tasks through structured communication (a reflection of stable cognitive patterns and qualitative representations that persist internally). Finally, we emphasized how the repetitive application of the KSTAR cycle, in a structured manner, leads to the emergence of stable cognitive patterns—recurrent solution templates, robust concepts, and a well-organized memory that together ensure the agent’s behavior becomes more refined and reliable with time.

In conclusion, the NEOLAF/KSTAR architecture demonstrates that by marrying rigorous theoretical principles with modern AI techniques, we can inch closer to artificial agents that learn autonomously and continuously from their own lived experience. The formalism of categories and functors guarantees that our design is not just a bag of tricks, but a cohesive system with properties like compositionality and naturality that are essential for scalability and robustness. Meanwhile, the use of large-scale neural learners embedded in a symbolic framework ensures that the agent can handle the richness of the real world—its uncertainty, diversity, and open-endedness—without sacrificing the benefits of clarity, modularity, and explainability. This theoretical foundation paves the way for the detailed algorithmic and empirical chapters to follow, wherein we will explore implementation details, experimental results, and comparisons with other cognitive architectures. The hope is that this synthesis of philosophical insight, mathematical rigor, and computational pragmatism offers a viable path toward truly never-ending open learning agents.

References

- [1] J. R. Anderson. An integrated theory of the mind. *Psychological review*, 111(4):1036, 2004.
- [2] A. S. d’Avila Garcez, L. C. Lamb, and M. Gori. Neural-symbolic learning and reasoning: An introduction and review. *Machine Learning*, 108(3):395–425, 2019.
- [3] J. E. Laird. *The Soar cognitive architecture*. MIT press, 2012.
- [4] J. Mao, C. Gan, P. Kohli, J. B. Tenenbaum, and J. Wu. The neuro-symbolic concept learner: Interpreting scenes, words, and sentences from natural supervision. In *International conference on learning representations*, 2019.
- [5] T. M. Mitchell. Never-ending learning. *Communications of the ACM*, 61(5):103–115, 2018.
- [6] R. S. Sutton and A. G. Barto. *Reinforcement learning: An introduction*. MIT press, 2018.
- [7] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. In *Advances in neural information processing systems*, pages 5998–6008, 2017.
- [8] Fei-Yue Wang, Xiao Wang, Lingxi Li, and Li Li. Steps toward parallel intelligence. *IEEE/CAA Journal of Automatica Sinica*, 3(4):345–348, 2016.
- [9] Jiacun Wang, Ying Tang, Ryan Hare, and Fei-Yue Wang. Parallel intelligent education with chatgpt. *Frontiers of Information Technology & Electronic Engineering*, 25:12–18, 2024.